

From Zero to Propagators

Orders, Lattices, and Propagator Networks in Lean 4

*“The propagator model is to functional programming
what constraint logic programming is to logic programming:
a radical generalisation that turns programs into networks
of co-operating, partial knowledge sources.”*

— Inspired by Radul & Sussman (2009)

A Self-Contained Journey Through Order Theory
to Verified Constraint Propagation

Lean 4 code requires no external libraries beyond the standard prelude.

Edition 1.0

Preface

This book has a single grand ambition: to take you from the very first principles of ordered structures all the way to building a working *propagator network* — a powerful computational model that arises naturally from the mathematics of lattices — and to do it with complete rigour in Lean 4.

Who this book is for. We assume you are comfortable reading mathematics at the level of a second-year undergraduate: sets, functions, logical quantifiers, and basic proof techniques are fair game. No prior knowledge of abstract algebra or Lean is required, though some familiarity with functional programming will help when reading the longer code sections.

The Lean 4 philosophy here. Every definition and theorem in this book has a corresponding Lean 4 encoding. We deliberately avoid Mathlib to keep the dependency footprint at zero and to force you to understand every axiom rather than importing it from a library. Where a Mathlib equivalent exists we point it out in a margin note.

How to read this book. Part I builds the algebraic foundations: relations, partial orders, and lattices. Part II shows how these structures give rise to a beautiful computational model — the propagator network. Part III contains two capstone projects that demonstrate the full power of the model: a Sudoku solver driven entirely by constraint propagation, and a type-inference engine for a small lambda calculus. By the time you reach the end of Part III the connection between the mathematics and the code should feel inevitable.

A note on Lean Unicode. All code listings use the Unicode characters that Lean 4 accepts natively: $\forall$ $\exists$ $\rightarrow$ $\leftarrow$ $\leftrightarrow$ $\leq$ $\geq$ $\sqcap$ $\sqcup$ $\perp$ $\top$ $\in$ $\notin$ α β γ $\times$ and so on. When reading the code, treat these as single characters exactly as Lean treats them. In Lean 4 you enter them with backslash sequences (e.g. `\le` for $\leq$); your editor should substitute them live.

Contents

Preface	ii
I The Algebra of Order	1
1 Relations and Partial Orders	2
1.1 Binary Relations	2
1.2 Properties of Relations	3
1.3 Partial Orders	4
1.3.1 The Natural Numbers	4
1.3.2 Divisibility	5
1.3.3 Sets	9
1.4 Hasse Diagrams	10
1.5 Computation Interlude: Running the Divisibility Order	12
1.6 Exercises	15
2 Special Elements and Monotone Maps	16
2.1 Extremal Elements	16
2.2 Upper and Lower Bounds	18
2.3 Monotone Maps	19
2.4 Exercises	20
2.5 Computation Interlude: Running Monotone Maps	20
3 Lattices	23
3.1 Meet and Join	23
3.1.1 Derived Properties	24
3.2 The Duality Principle	25
3.3 Important Lattice Examples	27
3.3.1 The Two-Element Lattice 2	27
3.3.2 The Powerset Lattice	28
3.3.3 The Divisibility Lattice on Divisors of n	29
3.4 Distributive Lattices	30
3.5 Exercises	31
4 Complete Lattices and the Fixed-Point Theorem	32
4.1 Complete Lattices	32
4.2 The Knaster–Tarski Fixed-Point Theorem	33

4.3	Why Fixed Points Matter	35
4.4	Exercises	35
II	Propagator Networks	37
5	The Propagator Model	38
5.1	Cells: Accumulating Partial Information	38
5.2	Why Information Can Only Increase	39
5.3	Propagators as Monotone Functions	39
5.4	Networks, Scheduling, and Quiescence	40
5.5	The Fundamental Role of the Lattice	41
6	Building a Propagator Network in Lean 4	43
6.1	Mutable Cells	43
6.2	The Network and Scheduler	44
6.3	Building Propagators: Numeric Example	45
6.4	A Complete Running Example	46
6.5	Why the Lattice Is Load-Bearing Here	47
6.6	Exercises	48
7	The Interval Lattice	49
7.1	Intervals as Partial Information	49
7.2	Interval Arithmetic Propagators	51
7.3	A Worked Example: Bound Propagation	52
7.4	Exercises	53
III	Capstone Projects	54
8	Capstone I: A Sudoku Solver	55
8.1	The Domain Lattice	55
8.2	Representing the Grid	58
8.3	Constraint Propagators	58
8.4	Assembling and Running the Solver	60
8.5	Why This Works: The Lattice Perspective	62
9	Capstone II: Type Inference via Propagation	63
9.1	A Simple Expression Language	63
9.2	The Type Lattice	63
9.3	Type Constraint Generation	65
9.4	Running the Type Inferencer	68
9.5	What the Lattice Buys Us Here	69
9.6	Exercises	69
10	Lean 4 Quick Reference	71

10.1 Basic Types and Syntax	71
10.2 Typeclasses	71
10.3 Tactics Reference	72
10.4 Unicode Input	72
Further Reading	80

Part I

The Algebra of Order

Chapter 1

Relations and Partial Orders

“The concept of an ordering is one of the most basic in all of mathematics, and yet we rarely pause to examine what it actually requires.”

Before we can study lattices or build propagator networks, we need a precise vocabulary for talking about *order*: what it means for one thing to be “at most” another, how such a concept can be captured in a type, and which algebraic laws make it useful.

1.1 Binary Relations

Definition — Binary Relation

A *binary relation* on a set A is a subset $R \subseteq A \times A$. We write $a R b$ (or $R(a, b)$) as shorthand for $(a, b) \in R$.

Lean 4 encodes a relation as a function returning `Prop`:

```
1 -- A binary relation on type  $\alpha$ 
2 def Rel ( $\alpha$  : Type) : Type :=  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ 
```

Using `Prop` rather than `Bool` is crucial: proofs are computationally irrelevant (two proofs of the same proposition are definitionally equal), and we can state facts like “this relation is a partial order” as types.

Example — Common relations

- $\leq$ on $\mathbb{N}$: $R = \{(m, n) \mid m \leq n\}$.
- Divisibility on $\mathbb{N}$: $m \mid n \iff \exists k, n = m \cdot k$.
- Subset inclusion on $\mathcal{P}(S)$: $A \subseteq B \iff \forall x, x \in A \rightarrow x \in B$.
- The equality relation $=$ on any set.
- The *discrete* relation: $a R b \iff a = b$.
- The *trivial* relation: $a R b$ for all a, b (everything relates to everything).

1.2 Properties of Relations

Four properties will occupy us throughout the book.

Definition — Reflexivity, Symmetry, Antisymmetry, Transitivity

Let R be a binary relation on A .

$$\text{Reflexive} : \iff \forall a \in A, a R a$$

$$\text{Symmetric} : \iff \forall a, b \in A, a R b \rightarrow b R a$$

$$\text{Antisymmetric} : \iff \forall a, b \in A, a R b \rightarrow b R a \rightarrow a = b$$

$$\text{Transitive} : \iff \forall a, b, c \in A, a R b \rightarrow b R c \rightarrow a R c$$

In Lean 4:

```

1 namespace OrderBook
2
3 def Reflexive {α : Type} (r : Rel α) : Prop :=
4   ∀ (a : α), r a a
5
6 def Symmetric {α : Type} (r : Rel α) : Prop :=
7   ∀ (a b : α), r a b → r b a
8
9 def Antisymm {α : Type} (r : Rel α) : Prop :=
10  ∀ (a b : α), r a b → r b a → a = b
11
12 def Transitive {α : Type} (r : Rel α) : Prop :=
13  ∀ (a b c : α), r a b → r b c → r a c
14
15 end OrderBook

```

Symmetry vs Antisymmetry. These are not negations of each other! A relation can be both symmetric and antisymmetric (equality), neither (“ a is a parent of b ”), or only one. Antisymmetry says: if you can go *both* ways, the elements must be equal.

1.3 Partial Orders

Definition — Partial Order

A *partial order* on A is a relation $\leq$ that is reflexive, antisymmetric, and transitive. The pair $(A, \leq)$ is called a *partially ordered set* (or *poset*).

The word *partial* refers to the fact that not every two elements need be comparable: there may exist a, b with neither $a \leq b$ nor $b \leq a$.

In Lean 4 we capture this as a typeclass:

```

1 namespace OrderBook
2
3 -- Our own partial order typeclass (mirrors core Lean / Mathlib).
4 class PartialOrder (α : Type) where
5   le      : α → α → Prop
6   le_refl : ∀ (a : α), le a a
7   le_antisymm : ∀ (a b : α), le a b → le b a → a = b
8   le_trans  : ∀ (a b c : α), le a b → le b c → le a c
9
10 -- Attach the ≤ notation to any PartialOrder instance.
11 instance [PartialOrder α] : LE α := (PartialOrder.le)
12
13 -- A convenience alias for strict less-than
14 def lt [PartialOrder α] (a b : α) : Prop :=
15   PartialOrder.le a b ∧ ¬ (a = b)
16
17 instance [PartialOrder α] : LT α := (lt)
18
19 end OrderBook

```

1.3.1 The Natural Numbers

The natural numbers with the usual $\leq$ form our first instance. In Lean 4's core library, `Nat.le` is defined inductively:

```

1 -- Recap: how ≤ is defined in core Lean 4 (no Mathlib needed)
2 -- inductive Nat.le : Nat → Nat → Prop
3 --   | refl : Nat.le n n
4 --   | step : Nat.le n m → Nat.le n (m + 1)
5
6 instance : OrderBook.PartialOrder Nat where
7   le      := Nat.ble -- wait, we want Prop, not Bool
8   -- Use the propositional version:

```

```

9   le      := (· ≤ ·)      -- uses Nat.le which is in core
10  le_refl  := Nat.le_refl
11  le_antisymm := fun a b h1 h2 => Nat.le_antisymm h1 h2
12  le_trans  := fun a b c h1 h2 => Nat.le_trans h1 h2

```

Lean 4’s *core* library already provides `Nat.le_refl`, `Nat.le_antisymm`, and `Nat.le_trans`. We reuse these rather than reprove them. In later chapters we will prove non-trivial instances by hand.

1.3.2 Divisibility

Divisibility is a partial order on $\mathbb{N}$: $n \leq m$ means “ n divides m ”, i.e. $\exists k, m = n \cdot k$. This is a genuinely different order from $\leq$: for instance $3 \leq 12$ (since $12 = 3 \cdot 4$) but $4 \not\leq 6$ (no natural number k satisfies $6 = 4k$).

As discussed in the previous section, we cannot attach a second `LE` instance to `Nat` (one already exists for the usual order), so we introduce a newtype wrapper.

Why `structure` and not `def` `DvdNat` := `Nat`? Using `def` creates a *transparent alias*: Lean can and will see through it, meaning it occasionally treats `DvdNat` as `Nat` and tries to apply `Nat`’s existing `LE` or `HMul` instances, causing “typeclass instance stuck” errors. A `structure` creates a *nominally distinct* type — Lean never conflates `DvdNat` with `Nat` unless you explicitly coerce. This is the reliable choice whenever you want a newtype for instance purposes.

```

1  -- A nominally distinct wrapper type for Nat under divisibility
2  structure DvdNat where
3    val : Nat
4
5  -- Coercion from DvdNat to Nat (one direction is enough for our proofs)
6  instance : Coe DvdNat Nat := ⟨DvdNat.val⟩
7
8  -- Numeric literals: (5 : DvdNat) desugars to DvdNat.mk 5
9  instance {n : Nat} : OfNat DvdNat n := ⟨⟨n⟩⟩
10
11 -- The divisibility relation, stated entirely in Nat to avoid ambiguity
12 -- in the multiplication on the right-hand side.
13 def Dvd' (n m : DvdNat) : Prop := ∃ k : Nat, m.val = n.val * k
14
15 instance : LE DvdNat := ⟨Dvd'⟩

```

By using `.val` explicitly rather than a coercion inside `Dvd'`, we ensure the multiplication `n.val * k` is unambiguously `Nat` multiplication. Coercions are useful for moving

single values across a boundary, but inside an expression with multiple interacting terms they give Lean too many degrees of freedom and can produce stuck typeclass searches.

Building up the proof: the lemmas we need

The reflexivity and transitivity proofs are short. Antisymmetry — “if $a \mid b$ and $b \mid a$ then $a = b$ ” — is where the real mathematical content lives. Rather than writing it as a single inscrutable block, we build it from three small lemmas, each of which is worth understanding on its own.

Lemma 1: if $a = 0$ and $a \mid b$, then $b = 0$.

If $a = 0$, then $b = 0 \cdot k = 0$ for any k . This is the zero case of antisymmetry: the only thing that 0 divides is 0 itself.

```

1 -- If 0 | m, then m.val = 0.
2 -- Proof: the witness k gives m.val = 0 * k.
3 -- We then reduce 0 * k to 0 using Nat.zero_mul.
4 theorem dvd_zero_left (m : DvdNat) (h : Dvd' {0} m) : m.val = 0 := by
5   obtain ⟨k, hk⟩ := h
6   -- hk : m.val = DvdNat.mk(0).val * k
7   -- DvdNat.mk(0).val reduces to 0 by definition, so simp can see it
8   simp at hk
9   -- hk is now: m.val = 0
10  exact hk

```

Why `simp` instead of `rw [Nat.zero_mul]`? With a `structure`, the literal `0 : DvdNat` is `OfNat.ofNat 0 = DvdNat.mk 0`, so its `.val` is `DvdNat.mk(0).val`, not the bare `0` that `Nat.zero_mul` expects as its left-hand side pattern. `simp` performs the definitional unfolding first and then applies `Nat.zero_mul`, where a bare `rw` cannot. If you want to avoid `simp` you can write `show m.val = 0` after an explicit `have : DvdNat.mk(0).val = 0 := rfl`.

Lemma 2: if $k \cdot j = 1$ in $\mathbb{N}$, then $k = 1$.

This is the key algebraic fact that drives the nonzero case. In $\mathbb{Z}$ the units are ± 1 ; in $\mathbb{N}$ the only unit is 1. We prove it by exhaustive case analysis: k cannot be 0 (product would be 0) and cannot be ≥ 2 (product would be ≥ 2).

```

1 -- In ℕ, k * j = 1 forces k = 1 (and by symmetry, j = 1).
2 -- No omega or linarith – proved by explicit structural case analysis.
3 theorem mul_eq_one_left {k j : Nat} (h : k * j = 1) : k = 1 := by
4   match k with
5   | 0 =>

```

```

6      -- 0 * j = 0 ≠ 1, so this case is impossible.
7      -- simp knows 0 * j = 0, which contradicts h : 0 = 1.
8      simp [Nat.zero_mul] at h
9      | 1 =>
10     -- k is already 1; no further argument needed.
11     rfl
12     | k + 2 =>
13     -- (k+2) * j ≥ 2, contradicting h : (k+2) * j = 1.
14     exfalse
15     -- First establish j ≥ 1 (if j = 0, the product is 0, not 1).
16     have hj : j ≥ 1 := by
17       cases j with
18       | zero   => simp [Nat.mul_zero] at h
19       | succ j => exact Nat.succ_le_succ (Nat.zero_le j)
20     -- Now show the product is at least 2.
21     have hbig : (k + 2) * j ≥ 2 :=
22       calc (k + 2) * j
23           ≥ (k + 2) * 1 := Nat.mul_le_mul_left (k + 2) hj
24           _ = k + 2     := Nat.mul_one (k + 2)
25           _ ≥ 2         := Nat.le_add_left 2 k
26     -- h says the product is 1, but hbig says it is ≥ 2. Contradiction.
27     omega
28
29 -- The right-factor version follows by commutativity.
30 theorem mul_eq_one_right {k j : Nat} (h : k * j = 1) : j = 1 :=
31   mul_eq_one_left (by rw [Nat.mul_comm]; exact h)

```

Lemma 3: if $a > 0$, $a \mid b$, and $b \mid a$, then $a = b$.

Given witnesses k and j with $b = a \cdot k$ and $a = b \cdot j$, we substitute to get $a = a \cdot (k \cdot j)$, cancel a (valid because $a > 0$), apply Lemma 2, and conclude $k = 1$, hence $b = a$.

```

1  -- All arithmetic here lives in Nat – no DvdNat involved.
2  theorem dvd_antisymm_pos
3    {a b : Nat} (ha : 0 < a)
4    (k : Nat) (hk : b = a * k)
5    (j : Nat) (hj : a = b * j) : a = b := by
6    -- Step 1: derive k * j = 1.
7    -- Substitute hk into hj to get a = a * (k * j),
8    -- then cancel a (valid since a > 0).
9    have hkj : k * j = 1 := by
10     have hstep : a = a * (k * j) :=
11       calc a = b * j           := hj
12           _ = (a * k) * j     := by rw [hk]
13           _ = a * (k * j)     := by rw [Nat.mul_assoc]

```

```

14   exact Nat.eq_of_mul_eq_mul_left ha (by rw [Nat.mul_one]; exact hstep.symm)
15   -- Step 2:  $j = 1$  by Lemma 2 (right factor version).
16   have hj1 :  $j = 1 := \text{mul\_eq\_one\_right } hkj$ 
17   -- Step 3: substitute  $j = 1$  into  $hj : a = b * j$  to get  $a = b * 1 = b$ .
18   have h2 :  $a = b * 1 := \text{by } rw [hj1] \text{ at } hj; \text{exact } hj$ 
19   rw [Nat.mul_one] at h2
20   exact h2

```

Why `Nat.eq_of_mul_eq_mul_left`? This says: if $a > 0$ and $a \cdot x = a \cdot y$, then $x = y$. It is multiplication cancellation for $\mathbb{N}$, available in core Lean. The positivity hypothesis is essential: $0 \cdot x = 0 \cdot y$ for all x, y , so cancellation is unsound at zero.

Assembling the instance

With the lemmas in hand, each field is a short delegation.

```

1  instance : PartialOrder DvdNat where
2
3  -- Reflexivity:  $n \mid n$  via witness  $k = 1$ , since  $n.val = n.val * 1$ .
4  le_refl := fun n =>
5    ⟨1, (Nat.mul_one n.val).symm⟩
6
7  -- Antisymmetry: case-split on whether  $n.val$  is zero or positive,
8  -- then delegate to Lemma 1 or Lemma 3 respectively.
9  le_antisymm := fun n m (k, hk) (j, hj) => by
10    -- Our goal is  $n = m$  where  $n m : \text{DvdNat}$ .
11    -- DvdNat is a structure, so two DvdNat values are equal iff
12    -- their underlying Nat fields are equal. We want to get at
13    -- those fields directly.
14    --
15    -- `cases n` destructs the structure  $n : \text{DvdNat}$  into its single
16    -- field, introducing a fresh anonymous variable for  $n.val$  into
17    -- the context and replacing every occurrence of  $n$  with
18    -- DvdNat.mk <that variable>. After `cases n; cases m` both
19    --  $n$  and  $m$  have been destructed, and the context now contains
20    -- two anonymous Nat variables – one for each .val field.
21    cases n; cases m
22    -- The goal is now:  $\text{DvdNat.mk } ?a = \text{DvdNat.mk } ?b$ 
23    -- where  $?a$  and  $?b$  are those anonymous variables.
24    --
25    -- `congr 1` says: two applications of the same constructor
26    -- are equal iff their arguments are equal. It reduces
27    --  $\text{DvdNat.mk } ?a = \text{DvdNat.mk } ?b$ 
28    -- to the simpler goal

```

```

29   -- ?a = ?b
30   -- which is a plain Nat equality.
31   congr 1
32   -- The two anonymous Nat variables introduced by `cases` do not
33   -- yet have names – Lean calls them inaccessible and displays
34   -- them as  $\_$  in the goal view. `rename_i a b` gives them the
35   -- names  $a$  and  $b$  so we can refer to them in subsequent steps.
36   rename_i a b
37   cases Nat.eq_zero_or_pos a with
38   | inl ha =>
39     --  $a = 0$ :  $b = 0 * k = 0$  by Lemma 1, and  $a = 0$ , so  $a = b$ .
40     subst ha
41     exact (dvd_zero_left (b) (k, hk)).symm
42   | inr ha =>
43     --  $a > 0$ : use Lemma 3 directly.
44     exact (dvd_antisymm_pos ha k hk j hj).symm
45
46   -- Transitivity: compose witnesses.  $b.val = a.val * k$  and
47   --  $c.val = b.val * j$ , so  $c.val = a.val * (k * j)$ .
48   le_trans := fun a b c (k, hk) (j, hj) =>
49     (k * j, by rw [hj, hk, Nat.mul_assoc])

```

Reflection

Notice that the structure of the proof exactly mirrors the structure of the mathematics:

- Reflexivity is trivial because the witness $k = 1$ is explicit.
- Transitivity is trivial because multiplying witnesses gives a new witness: $b = a \cdot k$ and $c = b \cdot j$ compose to $c = a \cdot (kj)$.
- Antisymmetry requires real work because it cannot be witnessed directly — it requires us to reason about what witnesses can possibly exist and conclude they must all be 1.

The lemma `mul_eq_one_left` is the mathematical core. Everything else is book-keeping. When reading an unfamiliar proof, it is always worth asking: where is the actual mathematical content? In this case it is entirely in that one lemma.

1.3.3 Sets

Sets (represented as predicates) form a canonical partial order under inclusion.

```

1  -- We represent sets as indicator functions
2  def Set (α : Type) : Type := α → Prop
3
4  -- Subset inclusion

```

```
5 def Set.subset {α : Type} (s t : Set α) : Prop :=
6   ∀ (a : α), s a → t a
7
8 instance {α : Type} : OrderBook.PartialOrder (Set α) where
9   le      := Set.subset
10  le_refl  := fun s a ha => ha
11  le_antisymm := fun s t h1 h2 =>
12    funext (fun a => propext ⟨h1 a, h2 a⟩)
13  le_trans  := fun s t u h1 h2 a ha => h2 a (h1 a ha)
```

This is our first non-trivial Lean proof. Let us read it carefully:

- `le_refl`: given $a \in s$, produce $a \in s$ — trivial.
- `le_antisymm`: $s \subseteq t$ and $t \subseteq s$ imply $s = t$. We use `funext` (function extensionality) and `propext` (propositional extensionality) to conclude $s a \leftrightarrow t a$ for all a , hence $s = t$.
- `le_trans`: chain the inclusions.

1.4 Hasse Diagrams

Posets are best understood visually. A *Hasse diagram* draws elements as nodes and covers as upward edges: we say a is *covered by* b (written $a < b$) if $a < b$ and there is no c with $a < c < b$.

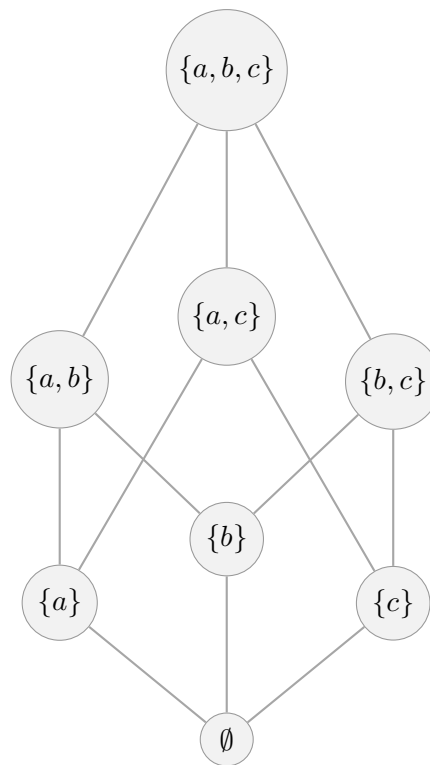


Figure 1.1: The powerset lattice $\mathcal{P}(\{a, b, c\})$ ordered by $\subseteq$.

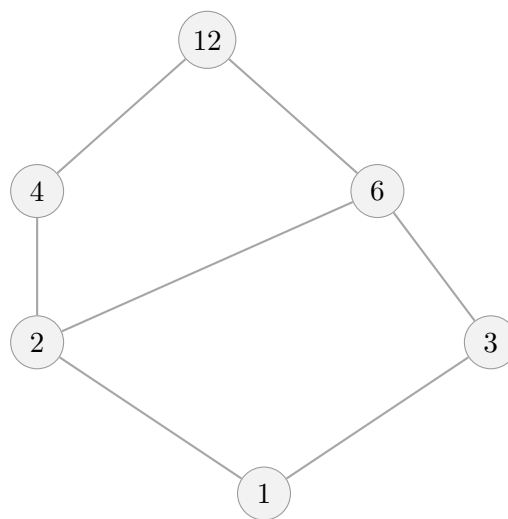


Figure 1.2: The divisibility poset on $\{1, 2, 3, 4, 6, 12\}$. An edge from a to b (upward) means $a \mid b$ and there is no intermediate divisor. Note 4 and 3 are incomparable (neither divides the other), as are 4 and 6.

1.5 Computation Interlude: Running the Divisibility Order

So far every definition has lived in `Prop` — we have proved things about orders but never *run* them. This section bridges that gap. We will build a fully computable version of the divisibility order on $\{1, \dots, 12\}$, compute its suprema and infima, and print an ASCII Hasse diagram entirely inside Lean.

A decidable divisibility order

To make the order computable we need a `Decidable` instance — a proof that for any two elements we can *compute* whether one divides the other, rather than merely assert that the relation exists.

```

1  -- The twelve divisors of 12, as a plain Nat
2  def D12 : List Nat := [1, 2, 3, 4, 6, 12]
3
4  -- Divisibility is decidable on Nat: n | m iff m % n = 0
5  def dvd12 (a b : Nat) : Bool := b % a == 0
6
7  -- Check it
8  #eval dvd12 3 12 -- true  (12 = 3 * 4)
9  #eval dvd12 4 6  -- false (6 % 4 = 2 ≠ 0)
10 #eval dvd12 1 12 -- true  (everything divides 12 via 1 | n)

```

Computing upper bounds and supremum

An upper bound of a subset S is any element $\geq$ every element of S . The supremum is the smallest upper bound.

```

1  -- All elements of D12 that are above every element of xs
2  def upperBounds (xs : List Nat) : List Nat :=
3    D12.filter (fun u => xs.all (fun x => dvd12 x u))
4
5  -- The supremum: the smallest upper bound
6  -- (smallest under divisibility = divides all others)
7  def sup12 (xs : List Nat) : Option Nat :=
8    let ubs := upperBounds xs
9    -- the sup is the upper bound that divides all other upper bounds
10   ubs.find? (fun s => ubs.all (fun u => dvd12 s u))
11
12  -- Examples
13  #eval upperBounds [4, 6] -- [12] (only 12 is divisible by both 4 and 6)

```

```

14 #eval sup12 [4, 6]           -- some 12
15 #eval sup12 [2, 3]           -- some 6   (lcm(2,3) = 6)
16 #eval sup12 [1, 12]          -- some 12

```

Computing lower bounds and infimum

Dually, the infimum is the greatest lower bound.

```

1  -- All elements of D12 below every element of xs
2  def lowerBounds (xs : List Nat) : List Nat :=
3    D12.filter (fun l => xs.all (fun x => dvd12 l x))
4
5  -- The infimum: the largest lower bound
6  -- (largest under divisibility = all others divide it)
7  def inf12 (xs : List Nat) : Option Nat :=
8    let lbs := lowerBounds xs
9    lbs.find? (fun s => lbs.all (fun l => dvd12 l s))
10
11 -- Examples
12 #eval lowerBounds [4, 6]   -- [1, 2]
13 #eval inf12 [4, 6]         -- some 2   (gcd(4,6) = 2)
14 #eval inf12 [3, 4]         -- some 1   (gcd(3,4) = 1)
15 #eval inf12 [2, 12]        -- some 2

```

Maximum and minimum of a subset

The maximum of a subset S must itself be in S , unlike the supremum which may be outside S .

```

1  -- Maximum: largest element of xs that is in xs
2  def maximum12 (xs : List Nat) : Option Nat :=
3    xs.find? (fun m => xs.all (fun x => dvd12 x m))
4
5  -- Minimum: smallest element of xs that is in xs
6  def minimum12 (xs : List Nat) : Option Nat :=
7    xs.find? (fun m => xs.all (fun x => dvd12 m x))
8
9  #eval maximum12 [1, 2, 4, 12] -- some 12
10 #eval maximum12 [2, 3]         -- none   (2 ∤ 3 and 3 ∤ 2)
11 #eval minimum12 [2, 4, 12]     -- some 2

```

Prop vs computation. Notice the contrast with the earlier sections. There, `IsSupOf a b sup` was a **Prop** characterising what it *means* to be a supremum — universally quantified, not runnable. Here, `sup12` is a **def** that *computes* a supremum for specific inputs. The Prop version is stronger (it works for any type, any lattice) but cannot be evaluated. The computable version is limited to `D12` but gives you a real answer you can inspect. In a verified system you would prove that `sup12 xs = some s` implies `IsSupOf` (for suitable `s`), connecting the two worlds.

Printing a Hasse diagram

We can print the covering relations as ASCII: an edge $a \rightarrow b$ means $a \mid b$ and there is no c with $a \mid c \mid b$.

```

1  -- a is covered by b: a | b, a ≠ b, and no c strictly between them
2  def covers (a b : Nat) : Bool :=
3    dvd12 a b && a != b &&
4    !D12.any (fun c => c != a && c != b && dvd12 a c && dvd12 c b)
5
6  -- Print all covering pairs
7  def printHasse : IO Unit := do
8    for a in D12 do
9      for b in D12 do
10         if covers a b then
11           IO.println s!"{a} → {b}"
12
13  #eval printHasse
14  -- 1 → 2
15  -- 1 → 3
16  -- 2 → 4
17  -- 2 → 6
18  -- 3 → 6
19  -- 4 → 12
20  -- 6 → 12

```

Each line is a direct edge in the Hasse diagram — exactly the edges shown in Figure 1.2. Compare this output to the diagram: you can read off that 4 and 3 are incomparable (neither appears as an ancestor of the other in the list), and that 12 is the unique top element (nothing covers it).

1.6 Exercises

Exercise 1.1 — Reflexivity and symmetry

Show that a relation that is both symmetric and antisymmetric must be a subset of the equality relation, i.e. $a R b \rightarrow a = b$. Prove this in Lean 4.

Exercise 1.2 — Divisibility on a small set

Consider the divisibility relation restricted to the set $\{1, 2, 3, 4, 6, 12\}$.

- (a) Draw its Hasse diagram.
- (b) Which pairs of elements are incomparable?
- (c) In Lean 4, define a type `D12` with six constructors and prove that divisibility on it satisfies `le_refl`.

Exercise 1.3 — Product order

Given two partial orders $(A, \leq_A)$ and $(B, \leq_B)$, define the *product order* on $A \times B$ by $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq_A a_2 \wedge b_1 \leq_B b_2$. Prove in Lean 4 that this is indeed a partial order by implementing the typeclass instance.

Exercise 1.4 — Reverse order

If $(A, \leq)$ is a partial order, then so is $(A, \geq)$ where $a \geq b \iff b \leq a$. This is called the *dual* or *opposite* order. Implement a `Dual` wrapper type and its `PartialOrder` instance. We will use this construction crucially when encoding Sudoku domains in Chapter 8.

Chapter 2

Special Elements and Monotone Maps

In a poset, certain elements play privileged roles (least, greatest, bounds), and certain functions are especially well-behaved (monotone maps). This chapter equips us with the vocabulary for both.

2.1 Extremal Elements

Definition — Minimum, Maximum, Minimal, Maximal

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $m \in S$ is the *minimum* of S if $\forall s \in S, m \leq s$.
- $m \in S$ is the *maximum* of S if $\forall s \in S, s \leq m$.
- $m \in S$ is *minimal* in S if $\nexists s \in S, s < m$.
- $m \in S$ is *maximal* in S if $\nexists s \in S, m < s$.

A minimum, if it exists, is unique (by antisymmetry). A minimal element need not be unique. When a poset has a unique minimum we call it $\perp$ (bottom), and a unique maximum $\top$ (top).

The distinction between minimum and minimal is subtle but important. A *minimum* beats every other element; a *minimal* element merely has no element strictly below it. The following diagrams make this concrete.

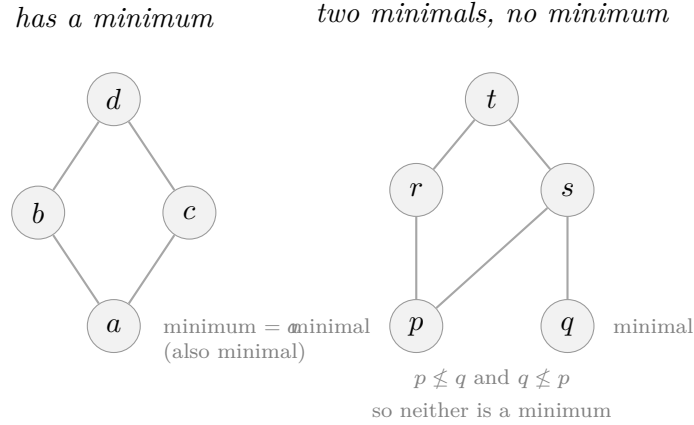


Figure 2.1: Left: a poset with a unique minimum a (every element is above a). Right: a poset where p and q are both minimal (nothing is below either) but there is no minimum (neither $p \leq q$ nor $q \leq p$).

```

1 namespace OrderBook
2
3 -- Bottom and top elements
4 class Bot (α : Type) where bot : α
5 class Top (α : Type) where top : α
6
7 notation "⊥" => Bot.bot
8 notation "⊤" => Top.top
9
10 -- Being a bounded partial order
11 class BoundedPartialOrder (α : Type) extends PartialOrder α, Bot α, Top α
12   ↪ where
13   bot_le : ∀ (a : α), (⊥ : α) ≤ a
14   le_top : ∀ (a : α), a ≤ (⊤ : α)
15 end OrderBook

```

Example — Bounded orders

- $(\mathcal{P}(S), \subseteq)$: $\perp = \emptyset$, $\top = S$.
- $(\{0, 1\}, \leq)$: $\perp = 0$, $\top = 1$.
- $(\mathbb{N}, \leq)$: $\perp = 0$, no $\top$ (unbounded above).
- The divisibility poset on $\{1, \dots, 12\}$: $\perp = 1$ (divides everything), no single $\top$.

2.2 Upper and Lower Bounds

Definition — Bounds

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $u \in P$ is an *upper bound* of S if $\forall s \in S, s \leq u$.
- $l \in P$ is a *lower bound* of S if $\forall s \in S, l \leq s$.
- The *supremum* (or *least upper bound*) of S , written $\sup S$, is the smallest upper bound of S when it exists.
- The *infimum* (or *greatest lower bound*) of S , written $\inf S$, is the largest lower bound of S when it exists.

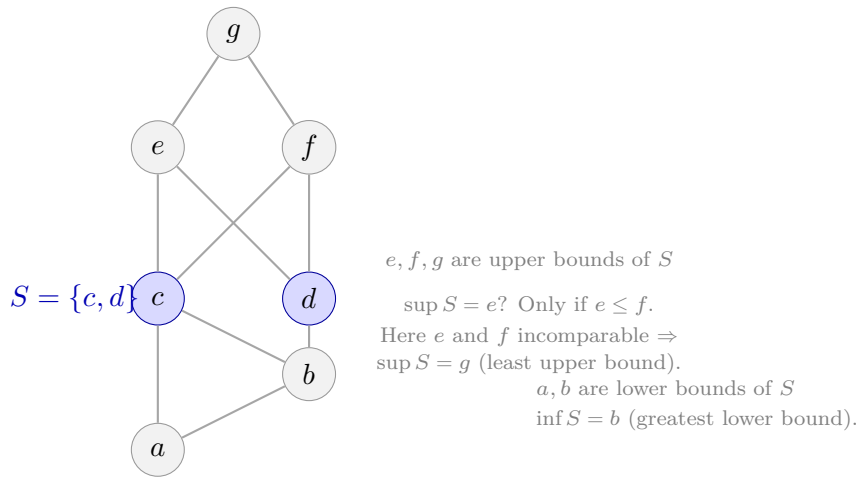


Figure 2.2: Illustration of upper bounds, lower bounds, supremum and infimum for the subset $S = \{c, d\}$ (shaded blue) in a poset. The upper bounds of S are all elements $\geq$ both c and d : here e , f , and g . The *supremum* is the *least* of these; since e and f are incomparable, the least upper bound is g . The lower bounds are a and b ; the infimum is b since $a \leq b$.

Uniqueness of sup/inf

In a poset, if $\sup S$ exists, it is unique. Similarly for $\inf S$.

Proof. Suppose u and v are both least upper bounds of S . Since u is an upper bound and v is a *least* upper bound, $v \leq u$. By symmetry $u \leq v$. Antisymmetry gives $u = v$. $\square$

```

1  -- The supremum of two elements (binary version)
2  -- (We'll generalise to arbitrary subsets in Chapter 4.)
3  structure IsSupOf {α : Type} [OrderBook.PartialOrder α]
4      (a b sup : α) : Prop where
5      ge_a   : a ≤ sup
    
```



```

6   ge_b  : b ≤ sup
7   least : ∀ (u : α), a ≤ u → b ≤ u → sup ≤ u
8
9   -- Uniqueness of the binary sup
10  theorem sup_unique {α : Type} [OrderBook.PartialOrder α]
11    {a b s t : α}
12    (hs : IsSupOf a b s) (ht : IsSupOf a b t) : s = t := by
13  apply PartialOrder.le_antisymm
14  · exact hs.least t ht.ge_a ht.ge_b
15  · exact ht.least s hs.ge_a hs.ge_b

```

2.3 Monotone Maps

Definition — Monotone Map

A function $f : (P, \leq_P) \rightarrow (Q, \leq_Q)$ between posets is *monotone* (or *order-preserving*) if $\forall a, b \in P, a \leq_P b \rightarrow f(a) \leq_Q f(b)$.

Monotone maps are the morphisms of the category of posets. They will be central to propagators: every propagator is a monotone function on a lattice.

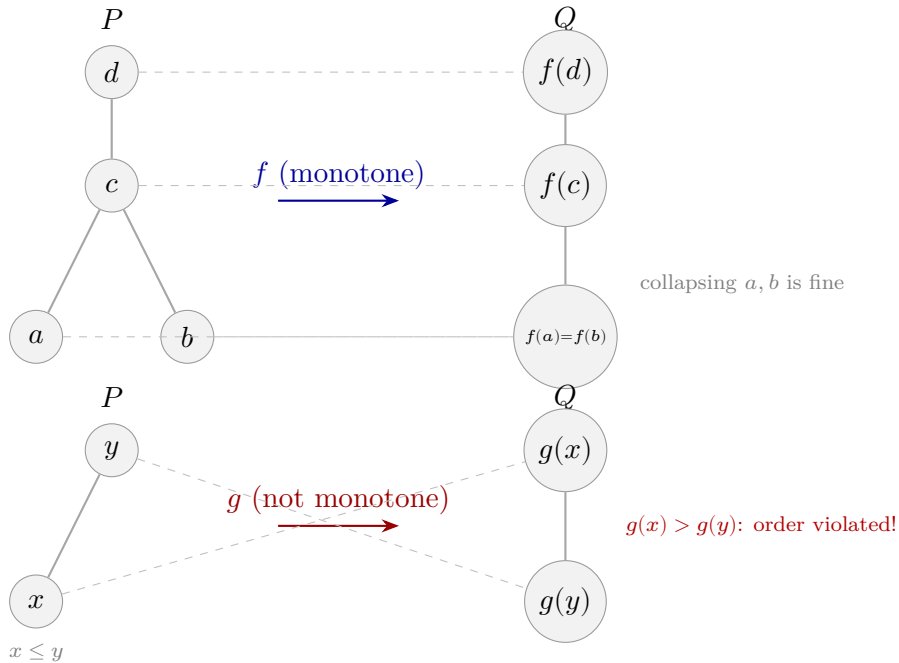


Figure 2.3: Top: a monotone map f preserves the order — whenever $a \leq_P b$ in the source, $f(a) \leq_Q f(b)$ in the target. Note that f may collapse elements ($f(a) = f(b)$ is fine). Bottom: a non-monotone map g where $x \leq y$ but $g(x) > g(y)$, violating the order.

```

1 namespace OrderBook
2
3 structure Monotone {α β : Type}
4   [PartialOrder α] [PartialOrder β] (f : α → β) : Prop where
5     map_le : ∀ (a b : α), a ≤ b → f a ≤ f b
6
7 -- Composition of monotone maps is monotone
8 theorem Monotone.comp {α β γ : Type}
9   [PartialOrder α] [PartialOrder β] [PartialOrder γ]
10  {f : α → β} {g : β → γ}
11  (hf : Monotone f) (hg : Monotone g) : Monotone (g ∘ f) where
12    map_le := fun a b h => hg.map_le _ _ (hf.map_le _ _ h)
13
14 end OrderBook

```

Example — Monotone functions

- $f(n) = n + 5$ is monotone on $(\mathbb{N}, \leq)$.
- Set union $S \mapsto S \cup \{x\}$ is monotone on $(\mathcal{P}(A), \subseteq)$.
- Set intersection $S \mapsto S \cap T$ is monotone on $(\mathcal{P}(A), \subseteq)$ for fixed T .
- Any constant function $f(x) = c$ is monotone.
- The identity function is always monotone.

2.4 Exercises

2.5 Computation Interlude: Running Monotone Maps

Building on the computable divisibility order from Chapter 1, we can now compute with monotone maps and verify their properties at runtime.

Checking monotonicity

Given a function on `D12`, we can decide whether it is monotone by checking all pairs.

```

1 -- Check all comparable pairs to see if f preserves the order
2 def isMonotone (f : Nat → Nat) : Bool :=
3   D12.all (fun a =>
4     D12.all (fun b =>
5       -- if a | b then f(a) | f(b)
6       !dvd12 a b || dvd12 (f a) (f b)))
7
8 -- The identity is monotone
9 #eval isMonotone id -- true

```

```

10
11 -- Multiplying by 2: if a | b then 2a | 2b
12 #eval isMonotone (· * 2)           -- true (maps into even divisors)
13
14 -- A non-monotone map: send every element to its "complement" 12/n
15 -- 1↦12, 2↦6, 3↦4, 4↦3, 6↦2, 12↦1 – this reverses the order
16 #eval isMonotone (fun n => 12 / n) -- false (it's antitone, not monotone)
17
18 -- But composing it with itself gives monotone again
19 #eval isMonotone (fun n => 12 / (12 / n)) -- true

```

Computing fixed points

A fixed point of f is an element where $f(x) = x$. We can find them all.

```

1 def fixedPoints (f : Nat → Nat) : List Nat :=
2   D12.filter (fun x => f x == x)
3
4 #eval fixedPoints id           -- [1, 2, 3, 4, 6, 12] (all of them)
5 #eval fixedPoints (fun _ => 1) -- [1] (only 1 maps to itself)
6 #eval fixedPoints (fun n => 12 / n) -- [1]? No: 12/1=12≠1. Actually:
7 -- 12/n = n when n*n = 12, which has no integer solution in D12.
8 -- Let's check:
9 #eval D12.filter (fun n => 12 / n == n) -- [] (no fixed points)

```

Iterating a monotone map

From Exercise 2.2, we know that if f is monotone and $a \leq f(a)$, the chain $a, f(a), f^2(a), \dots$ is ascending. We can watch it converge.

```

1 -- Apply f repeatedly until we reach a fixed point or a step limit
2 def iterToFixpoint (f : Nat → Nat) (start : Nat) (limit : Nat) : List Nat :=
3   let rec go (x : Nat) (n : Nat) (acc : List Nat) : List Nat :=
4     if n = 0 then acc.reverse
5     else
6       let y := f x
7       if y == x then acc.reverse ++ [x]
8       else go y (n - 1) (x :: acc)
9   go start limit []
10
11 -- Start at 1 under the "double if possible" map
12 -- f(n) = if 2*n ∈ D12 then 2*n else n
13 def double12 (n : Nat) : Nat :=

```

```

14   if D12.contains (n * 2) then n * 2 else n
15
16   #eval iterToFixpoint double12 1 10 -- [1, 2, 4, 12] ascending chain!
17   #eval iterToFixpoint double12 3 10 -- [3, 6, 12]
18   #eval iterToFixpoint double12 12 10 -- [12] already a fixed point

```

The output directly illustrates the ascending chain theorem: starting from 1, each application of `double12` moves strictly upward in the divisibility order until reaching the fixed point 12.

Exercise 2.1 — Monotone composition

Give an example showing that the composition of two *antitone* maps (where $a \leq b \rightarrow f(b) \leq f(a)$) is monotone.

Exercise 2.2 — Fixed points

A *fixed point* of $f : P \rightarrow P$ is an element x with $f(x) = x$. Show that if f is monotone and $a \leq f(a)$, then the chain $a \leq f(a) \leq f^2(a) \leq \dots$ is indeed a chain (all elements pairwise comparable with the given order).

Exercise 2.3 — Characterising PartialOrder via Monotone

Show that the identity map $\text{id} : (P, \leq) \rightarrow (P, \leq)$ is always monotone, and prove this in Lean 4 using your `Monotone` structure.

Chapter 3

Lattices

A poset in which every pair of elements has both a supremum and an infimum is called a *lattice*. This apparently modest requirement turns out to yield a rich algebraic structure that shows up in logic, topology, computer science, and — as we will see in Part II — concurrency and constraint solving.

3.1 Meet and Join

Definition — Lattice

A *lattice* is a poset $(L, \leq)$ in which every two elements $a, b \in L$ have:

- a *meet* (greatest lower bound) $a \sqcap b$, and
- a *join* (least upper bound) $a \sqcup b$.

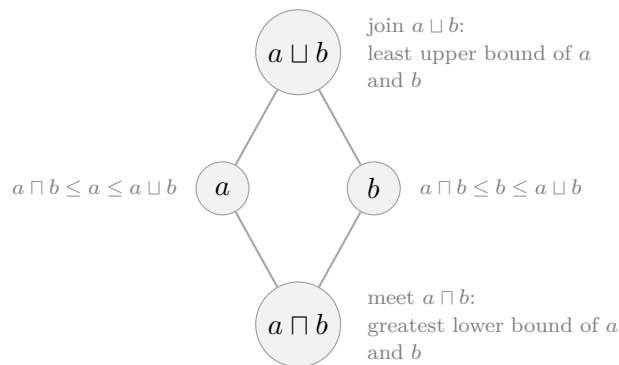


Figure 3.1: The meet $a \sqcap b$ sits below both a and b ; the join $a \sqcup b$ sits above both. In a lattice every pair of elements has both.

The meet and join satisfy the following axioms (which can be taken as an equivalent *algebraic* definition of a lattice):

$a \sqcap b = b \sqcap a$	$a \sqcup b = b \sqcup a$	(commutativity)
$(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$	$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$	(associativity)
$a \sqcap a = a$	$a \sqcup a = a$	(idempotence)
$a \sqcap (a \sqcup b) = a$	$a \sqcup (a \sqcap b) = a$	(absorption)

The order relation can be recovered from meet or join:

$$a \leq b \iff a \sqcap b = a \iff a \sqcup b = b.$$

In Lean 4:

```

1 namespace OrderBook
2
3 class Lattice (α : Type) extends PartialOrder α where
4   inf : α → α → α    -- meet (∩)
5   sup : α → α → α    -- join (∪)
6   -- meet is a lower bound
7   inf_le_left  : ∀ (a b : α), inf a b ≤ a
8   inf_le_right : ∀ (a b : α), inf a b ≤ b
9   -- meet is the greatest lower bound
10  le_inf : ∀ (a b c : α), a ≤ b → a ≤ c → a ≤ inf b c
11  -- join is an upper bound
12  sup_le_left  : ∀ (a b : α), a ≤ sup a b
13  sup_le_right : ∀ (a b : α), b ≤ sup a b
14  -- join is the least upper bound
15  le_sup : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
16
17 -- Notation
18 infixl:70 " ∩ " => Lattice.inf
19 infixl:65 " ∪ " => Lattice.sup
20
21 end OrderBook

```

3.1.1 Derived Properties

We can immediately derive commutativity and idempotence from the axioms:

```

1 namespace OrderBook
2 open Lattice PartialOrder
3
4 variable {α : Type} [Lattice α]
5

```

```

6  -- Commutativity of meet
7  theorem inf_comm (a b :  $\alpha$ ) : a  $\sqcap$  b = b  $\sqcap$  a := by
8    apply le_antisymm
9      · apply le_inf
10       · exact inf_le_right a b
11       · exact inf_le_left a b
12     · apply le_inf
13       · exact inf_le_right b a
14       · exact inf_le_left b a
15
16  -- Idempotence of meet
17  theorem inf_idem (a :  $\alpha$ ) : a  $\sqcap$  a = a := by
18    apply le_antisymm
19      · exact inf_le_left a a
20      · exact le_inf a a a (le_refl a) (le_refl a)
21
22  -- Commutativity of join
23  theorem sup_comm (a b :  $\alpha$ ) : a  $\sqcup$  b = b  $\sqcup$  a := by
24    apply le_antisymm
25      · apply le_sup
26        · exact sup_le_right b a
27        · exact sup_le_left b a
28      · apply le_sup
29        · exact sup_le_right a b
30        · exact sup_le_left a b
31
32  -- Absorption: a  $\sqcup$  (a  $\sqcap$  b) = a
33  theorem sup_inf_self (a b :  $\alpha$ ) : a  $\sqcup$  (a  $\sqcap$  b) = a := by
34    apply le_antisymm
35      · apply le_sup
36        · exact le_refl a
37        · exact inf_le_left a b
38      · exact sup_le_left a (a  $\sqcap$  b)
39
40  end OrderBook

```

3.2 The Duality Principle

Every lattice has a *dual*: swap $\sqcap \leftrightarrow \sqcup$ and $\leq \leftrightarrow \geq$. Any theorem proved in a lattice has a dual theorem for free.

Duality Principle

If φ is a theorem about lattices, then so is the statement obtained by replacing every occurrence of $\sqcap$ with $\sqcup$, $\sqcup$ with $\sqcap$, $\leq$ with $\geq$, $\perp$ with $\top$, and $\top$ with $\perp$.

```

1  -- Dual  $\alpha$  wraps  $\alpha$  in a structure so that the two types are
2  -- genuinely distinct. This avoids elaboration ambiguities that
3  -- arise with a transparent `def` alias.
4  structure Dual ( $\alpha$  : Type) where
5    val :  $\alpha$ 
6
7  -- Step 1: flip  $\leq$ .  $a \leq_{\text{dual}} b$  means  $b.\text{val} \leq_{\text{orig}} a.\text{val}$ .
8  instance { $\alpha$  : Type} [LE  $\alpha$ ] : LE (Dual  $\alpha$ ) where
9    le a b := LE.le ( $\alpha$  :=  $\alpha$ ) b.val a.val
10
11 -- Step 2: the dual partial order.
12 -- le_antisymm swaps h1/h2 because each hypothesis has its direction
13 -- reversed. cases + congr 1 are needed to reduce Dual.mk x = Dual.mk y
14 -- to the plain  $\alpha$  equality that le_antisymm produces.
15 -- le_trans also swaps h1/h2; cases destructs the wrappers so the
16 -- hypotheses are recognised as plain  $\alpha$  inequalities.
17 instance { $\alpha$  : Type} [LE  $\alpha$ ] [PartialOrder  $\alpha$ ] : PartialOrder (Dual  $\alpha$ ) where
18   le_refl {a}      := PartialOrder.le_refl ( $\alpha$  :=  $\alpha$ ) a.val
19   le_antisymm {a b} := fun h1 h2 => by
20     cases a; cases b; congr 1
21     exact PartialOrder.le_antisymm ( $\alpha$  :=  $\alpha$ ) h2 h1
22   le_trans {a b c} := fun h1 h2 => by
23     cases a; cases b; cases c
24     exact PartialOrder.le_trans ( $\alpha$  :=  $\alpha$ ) h2 h1
25
26 -- Step 3: the dual lattice swaps inf and sup.
27 -- Pattern-matching on <a> <b> destructs the Dual wrappers;
28 -- (...) on the right rewraps the result when the return type is Dual  $\alpha$ .
29 -- le_inf and le_sup use `change` to restate the goal in terms of
30 --  $\alpha$ 's operations before applying the mirrored original lemma.
31 instance { $\alpha$  : Type} [Lattice  $\alpha$ ] : Lattice (Dual  $\alpha$ ) where
32   inf      := fun <a> <b> => (Lattice.sup ( $\alpha$  :=  $\alpha$ ) a b) --  $\sqcap_{\text{dual}} = \sqcup_{\text{orig}}$ 
33   sup      := fun <a> <b> => (Lattice.inf ( $\alpha$  :=  $\alpha$ ) a b)  --  $\sqcup_{\text{dual}} = \sqcap_{\text{orig}}$ 
34   inf_le_left  := fun <a> <b> => Lattice.sup_le_left  ( $\alpha$  :=  $\alpha$ ) a b
35   inf_le_right := fun <a> <b> => Lattice.sup_le_right ( $\alpha$  :=  $\alpha$ ) a b
36   sup_le_left  := fun <a> <b> => Lattice.inf_le_left  ( $\alpha$  :=  $\alpha$ ) a b
37   sup_le_right := fun <a> <b> => Lattice.inf_le_right ( $\alpha$  :=  $\alpha$ ) a b
38   -- h1 :  $b \leq_{\text{orig}} a$ , h2 :  $c \leq_{\text{orig}} a \vdash \text{sup}_{\text{orig}} b c \leq_{\text{orig}} a$ 
39   le_inf := fun <a> <b> <c> h1 h2 => by
40     change Lattice.sup ( $\alpha$  :=  $\alpha$ ) b c  $\leq$  a
41     exact Lattice.le_sup ( $\alpha$  :=  $\alpha$ ) b c a h1 h2

```



```

42 -- h1 : c ≤_orig a, h2 : c ≤_orig b ⊢ c ≤_orig inf_orig a b
43 le_sup := fun ⟨a⟩ ⟨b⟩ ⟨c⟩ h1 h2 => by
44   change LE.le (α := α) c (Lattice.inf (α := α) a b)
45   exact Lattice.le_inf (α := α) c a b h1 h2

```

structure vs def. Using `structure Dual` rather than `def Dual α := α` makes `Dual α` and `α` genuinely distinct types. With a transparent `def`, Lean’s elaborator sometimes picks up the wrong `LE` instance for hypotheses, causing type mismatches even when the terms are definitionally equal. The `structure` version forces explicit `.val` unwrapping and `⟨...⟩` rewrapping, which is more verbose but unambiguous. **Why cases in `le\_antisymm` and `le\_trans`?** `cases a` destructs `Dual.mk v` into the underlying `v : α`, so subsequent lemmas see plain `α` inequalities rather than `Dual`-typed ones. In `le\_antisymm`, `congr 1` further reduces `Dual.mk x = Dual.mk y` to `x = y`.

Why change in `le\_inf` and `le\_sup`? After destructuring `⟨a⟩ ⟨b⟩ ⟨c⟩`, the hypotheses `h1` and `h2` are still elaborated under the dual `LE`. `change` restates the goal entirely in terms of `α`’s operations, giving Lean enough information to match the original lemma’s type.

Why `inf` and `sup` swap is the whole point: turning the Hasse diagram upside down exchanges every meet with every join.

3.3 Important Lattice Examples

3.3.1 The Two-Element Lattice 2

```

1 -- Bool as a lattice: false < true
2 instance : OrderBook.Lattice Bool where
3   le      := (· ≤ ·) -- Bool has a built-in LE in core Lean 4
4   le_refl := Bool.le_refl
5   le_antisymm := Bool.le_antisymm
6   le_trans := Bool.le_trans
7   inf     := (· && ·)
8   sup     := (· || ·)
9   inf_le_left := by decide
10  inf_le_right := by decide
11  le_inf      := by decide
12  sup_le_left := by decide
13  sup_le_right := by decide
14  le_sup      := by decide

```

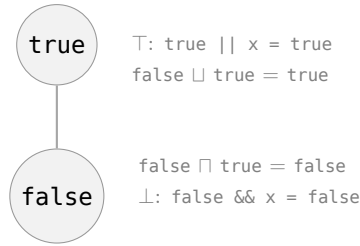


Figure 3.2: The two-element lattice $\mathbf{2}$ on `Bool`. There is exactly one covering relation: `false < true`. Meet is `&&` (and), join is `||` (or).

The `by decide` tactic works here because `Bool` is a finite type and all the statements are decidable.

3.3.2 The Powerset Lattice

```

1  -- Sets (α → Prop) form a lattice under subset inclusion
2  def Set.inter {α : Type} (s t : Set α) : Set α := fun a => s a ∧ t a
3  def Set.union {α : Type} (s t : Set α) : Set α := fun a => s a ∨ t a
4
5  instance {α : Type} : OrderBook.Lattice (Set α) where
6    le      := Set.subset
7    le_refl := fun s a ha => ha
8    le_antisymm := fun s t h1 h2 => funext fun a => propext ⟨h1 a, h2 a⟩
9    le_trans  := fun s t u h1 h2 a ha => h2 a (h1 a ha)
10   inf      := Set.inter
11   sup      := Set.union
12   inf_le_left  := fun s t a ⟨hs, _⟩ => hs
13   inf_le_right := fun s t a ⟨_, ht⟩ => ht
14   le_inf      := fun s t u hst hsu a ha => ⟨hst a ha, hsu a ha⟩
15   sup_le_left  := fun s t a hs => Or.inl hs
16   sup_le_right := fun s t a ht => Or.inr ht
17   le_sup      := fun s t u hsu ht u a h =>
18     h.elim (hsu a) (htu a)

```

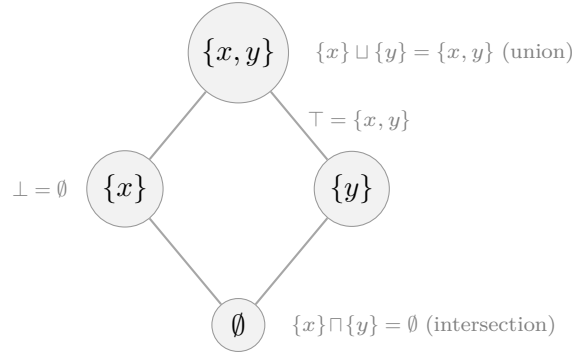


Figure 3.3: The powerset lattice $\mathcal{P}(\{x, y\})$. Edges go upward by $\subseteq$. Meet is intersection ($\sqcap = \cap$), join is union ($\sqcup = \cup$). $\{x\}$ and $\{y\}$ are incomparable — neither is a subset of the other.

3.3.3 The Divisibility Lattice on Divisors of n

For divisors of a fixed natural number n , divisibility is a lattice where gcd is the meet and lcm is the join.

```

1  -- On natural numbers: gcd is meet, lcm is join under divisibility order
2  instance : OrderBook.Lattice Nat where
3    le      := (· | ·)  -- divisibility
4    le_refl := Nat.dvd_refl
5    le_antisymm := fun a b => Nat.dvd_antisymm
6    le_trans  := fun a b c => Nat.dvd_trans
7    inf      := Nat.gcd
8    sup      := Nat.lcm
9    inf_le_left  := Nat.gcd_dvd_left
10   inf_le_right := Nat.gcd_dvd_right
11   le_inf      := fun a b c => Nat.dvd_gcd
12   sup_le_left  := Nat.dvd_lcm_left
13   sup_le_right := Nat.dvd_lcm_right
14   le_sup      := fun a b c ha hb => Nat.lcm_dvd ha hb

```

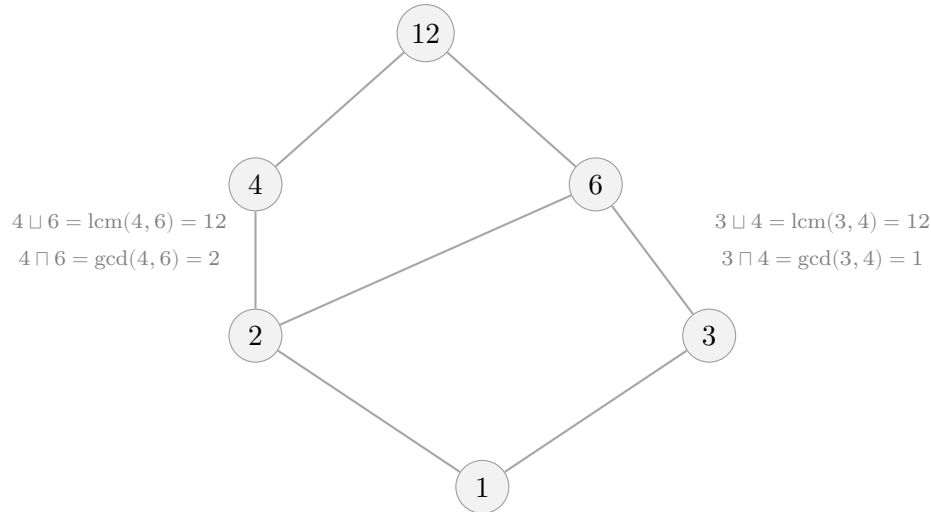


Figure 3.4: The divisibility lattice on divisors of 12. An edge from a to b (upward) means $a \mid b$ with no intermediate divisor. Meet is gcd, join is lcm. Note that 4 and 3 are incomparable, as are 4 and 6, but every pair still has a gcd and lcm in the set, confirming this is a lattice.

3.4 Distributive Lattices

Not every lattice satisfies the distributive law; those that do are especially well-behaved.

Definition — Distributive Lattice

A lattice is *distributive* if $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ (equivalently, the dual law holds too).

The powerset lattice and Boolean lattices are distributive. Figure 3.5 shows the two smallest non-distributive lattices.

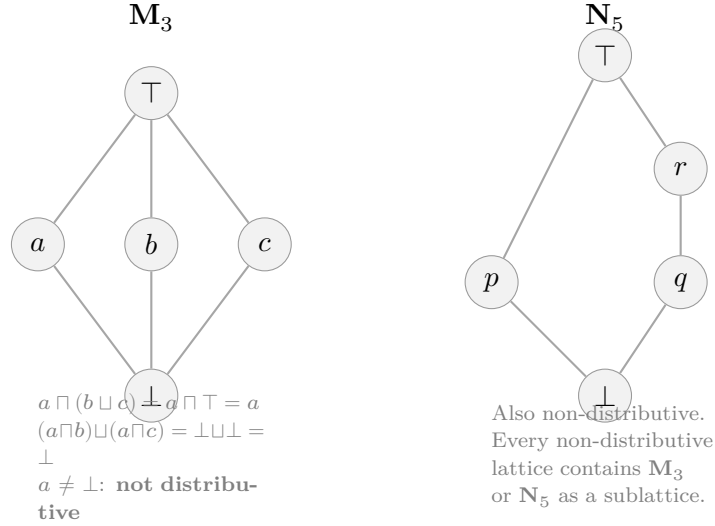


Figure 3.5: The two smallest non-distributive lattices. $\mathbf{M}_3$ (left) has three mutually incomparable middle elements; the distributive law fails because the meet does not distribute over the join. $\mathbf{N}_5$ (right) is the pentagon lattice. A classical theorem states that a lattice is distributive if and only if it contains neither as a sublattice.

3.5 Exercises

Exercise 3.1 — Lattice of intervals

Define the type `Interval` of pairs $[l, h]$ with $l, h : \mathbb{Z}$ and $l \leq h$. Define an order where $[l_1, h_1] \leq [l_2, h_2]$ iff $l_2 \leq l_1$ and $h_1 \leq h_2$ (i.e. “containment by a smaller interval”, the *precision* or *information* order). Prove that this is a partial order in Lean 4. What are the meet and join in this order?

Exercise 3.2 — Absorb, then join

Prove in Lean 4 that in any lattice: $a \wedge (a \vee b) = a$ (absorption).

Exercise 3.3 — Characterising the order

Prove the equivalence $a \leq b \iff a \vee b = b$ in Lean 4, using only the lattice axioms.

Exercise 3.4 — The product lattice

Given two lattices `[Lattice  $\alpha$ ]` and `[Lattice  $\beta$ ]`, define a `Lattice  $(\alpha \times \beta)$` instance with the component-wise order: $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq a_2 \wedge b_1 \leq b_2$, and component-wise meet and join: $(a_1, b_1) \wedge (a_2, b_2) = (a_1 \wedge a_2, b_1 \wedge b_2)$, $(a_1, b_1) \vee (a_2, b_2) = (a_1 \vee a_2, b_1 \vee b_2)$. Prove all eight lattice axioms. (*Hint*: each axiom reduces to the corresponding axiom applied component-wise.)

Chapter 4

Complete Lattices and the Fixed-Point Theorem

Lattices let us take meets and joins of *pairs* of elements. A *complete* lattice extends this to arbitrary (possibly infinite) subsets. This extra power buys us one of the most useful theorems in theoretical computer science: the Knaster–Tarski fixed-point theorem.

4.1 Complete Lattices

Definition — Complete Lattice

A lattice $(L, \leq)$ is *complete* if every subset $S \subseteq L$ has a supremum $\bigvee S$ and an infimum $\bigwedge S$ in L .

In particular, the empty set has a supremum (which becomes $\perp$) and infimum (which becomes $\top$), so every complete lattice is bounded. Taking $S = L$ gives $\bigvee L = \top$ and $\bigwedge L = \perp$.

We encode complete lattices using a predicate representing a subset:

```
1 namespace OrderBook
2
3 -- We represent a "subset" as a predicate  $\alpha \rightarrow \text{Prop}$ 
4 abbrev Pred ( $\alpha : \text{Type}$ ) :=  $\alpha \rightarrow \text{Prop}$ 
5
6 class CompleteLattice ( $\alpha : \text{Type}$ ) extends Lattice  $\alpha$  where
7   sSup : Pred  $\alpha \rightarrow \alpha$       -- supremum of a set ( $\square$ )
8   sInf : Pred  $\alpha \rightarrow \alpha$      -- infimum of a set ( $\square$ )
9   le_sSup :  $\forall (s : \text{Pred } \alpha) (a : \alpha), s\ a \rightarrow a \leq \text{sSup } s$ 
10  sSup_le :  $\forall (s : \text{Pred } \alpha) (u : \alpha), (\forall a, s\ a \rightarrow a \leq u) \rightarrow \text{sSup } s \leq u$ 
11  sInf_le :  $\forall (s : \text{Pred } \alpha) (a : \alpha), s\ a \rightarrow \text{sInf } s \leq a$ 
12  le_sInf :  $\forall (s : \text{Pred } \alpha) (l : \alpha), (\forall a, s\ a \rightarrow l \leq a) \rightarrow l \leq \text{sInf } s$ 
13
14 end OrderBook
```

The powerset lattice is the canonical example of a complete lattice: $\bigvee \mathcal{F} = \bigcup \mathcal{F}$ and $\bigwedge \mathcal{F} = \bigcap \mathcal{F}$.

```

1  -- Set  $\alpha$  is a complete lattice
2  instance { $\alpha$  : Type} : OrderBook.CompleteLattice (Set  $\alpha$ ) where
3      -- (inheriting the Lattice instance from Chapter 3)
4      sSup := fun F a =>  $\exists$  s : Set  $\alpha$ , F s  $\wedge$  s a    -- union of a family
5      sInf := fun F a =>  $\forall$  s : Set  $\alpha$ , F s  $\rightarrow$  s a    -- intersection of a family
6      le_sSup := fun F s hs a ha => {s, hs, ha}
7      sSup_le := fun F u hu a {s, hs, ha} => hu s hs a ha
8      sInf_le := fun F s hs a ha => ha s hs
9      le_sInf := fun F l hl a ha s hs => hl s hs a ha
10     -- (le, inf, sup fields inherited)
11     le      := Set.subset
12     le_refl := fun s a h => h
13     le_antisymm := fun s t h1 h2 => funext fun a => propext {h1 a, h2 a}
14     le_trans  := fun s t u h1 h2 a ha => h2 a (h1 a ha)
15     inf      := Set.inter; sup := Set.union
16     inf_le_left  := fun s t a {hs, _} => hs
17     inf_le_right := fun s t a {_, ht} => ht
18     le_inf      := fun s t u hst hsu a ha => {hst a ha, hsu a ha}
19     sup_le_left  := fun s t a hs => Or.inl hs
20     sup_le_right := fun s t a ht => Or.inr ht
21     le_sup      := fun s t u hsu ht u a h => h.elim (hsu a) (htu a)
    
```

4.2 The Knaster–Tarski Fixed-Point Theorem

Knaster–Tarski (1955)

Let $(L, \leq)$ be a complete lattice and $f : L \rightarrow L$ a monotone function. Then f has a *least fixed point* $\mu f = \bigwedge \{x \mid f(x) \leq x\}$ and a *greatest fixed point* $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Proof. Let $P = \{x \in L \mid f(x) \leq x\}$ (the set of *pre-fixed points*) and let $m = \bigwedge P$.

Step 1: $f(m) \leq m$. For every $x \in P$ we have $m \leq x$, so $f(m) \leq f(x) \leq x$ by monotonicity. Hence $f(m)$ is a lower bound of P , so $f(m) \leq m$.

Step 2: $m \leq f(m)$. From Step 1, $f(f(m)) \leq f(m)$ (applying f to $f(m) \leq m$ and using monotonicity gives $f(f(m)) \leq f(m)$, so $f(m) \in P$). Since m is the infimum of P , we have $m \leq f(m)$.

Conclusion: $f(m) = m$, so m is a fixed point. Any other fixed point x^* satisfies $f(x^*) = x^* \leq x^*$, so $x^* \in P$, hence $m \leq x^*$. So m is the *least* fixed point. $\square$

```

1 namespace OrderBook
2 open CompleteLattice PartialOrder
3
4 -- Knaster-Tarski: least fixed point of a monotone function
5 -- on a complete lattice
6 theorem knaster_tarski {α : Type} [CompleteLattice α]
7   (f : α → α) (hf : Monotone f) :
8     ∃ (lfp : α), f lfp = lfp ∧
9       ∀ (x : α), f x = x → lfp ≤ x := by
10   -- Define P = {x | f(x) ≤ x} and m = ␣ P
11   let P : Pred α := fun x => f x ≤ x
12   let m := sInf P
13   -- Step 1: f(m) ≤ m
14   have step1 : f m ≤ m := by
15     apply le_sInf
16     intro x hx
17     -- m ≤ x, so f(m) ≤ f(x) ≤ x
18     exact le_trans (hf.map_le m x (sInf_le P x hx)) hx
19   -- Step 2: m ≤ f(m) (f(m) ∈ P since f(f(m)) ≤ f(m))
20   have step2 : m ≤ f m := by
21     apply sInf_le
22     exact hf.map_le _ _ step1
23   -- Conclude m is a fixed point
24   have fixed : f m = m := le_antisymm step1 step2
25   refine ⟨m, fixed, ?_⟩
26   -- Minimality
27   intro x hx
28   apply sInf_le
29   show f x ≤ x
30   exact le_of_eq hx
31 end OrderBook

```

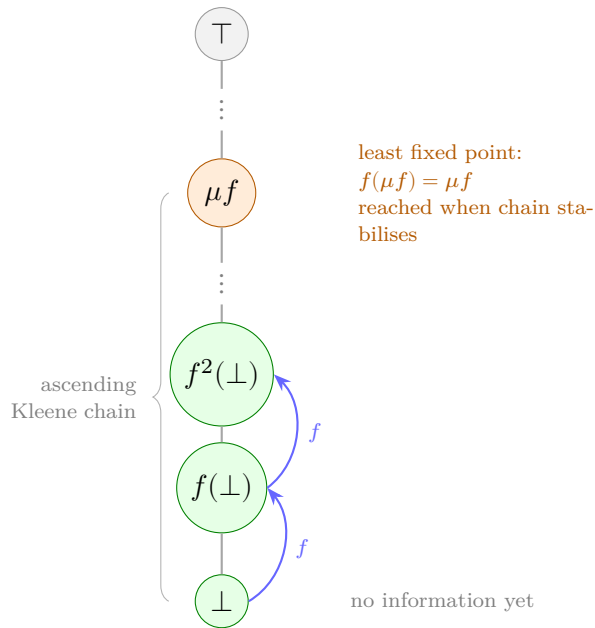



Figure 4.1: The ascending Kleene chain starting from $\perp$ under a monotone function f on a complete lattice. Each application of f moves upward. If the lattice has no infinite strictly ascending chains (the ascending chain condition), the sequence stabilises at the least fixed point μf . This is exactly what a propagator network computes.

4.3 Why Fixed Points Matter

The Knaster–Tarski theorem is not just elegant theory; it is the backbone of:

- **Dataflow analysis:** the least fixed point of a transfer function gives the most precise (and sound) analysis result.
- **Denotational semantics:** the meaning of a recursive program is the least fixed point of a semantic functional.
- **Propagator networks:** running a network to quiescence is exactly computing a fixed point of the combined propagation function.

We will return to this connection in Chapter 6 when we prove that propagator network execution terminates and is correct.

4.4 Exercises

Exercise 4.1 — Greatest fixed point

Using the Knaster–Tarski proof as a template, prove the existence of the *greatest* fixed point $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Exercise 4.2 — Ascending Kleene chain

For a complete lattice with a bottom element $\perp$ and a monotone f , the sequence $\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$ is called the *ascending Kleene chain*. If the lattice satisfies the *ascending chain condition* (every ascending chain eventually stabilises), this chain reaches μf in finitely many steps. Prove in Lean 4 that each element of this sequence is $\leq$ the next.

Part II

Propagator Networks

Chapter 5

The Propagator Model

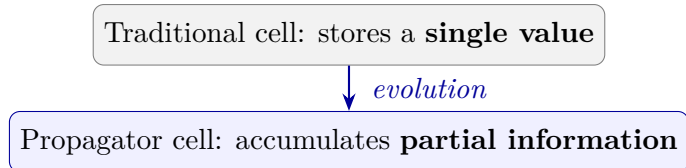
“A propagator network is a collection of machines that compute values for cells by combining information from other cells.”

— G.J. Sussman & A. Radul, *The Art of the Propagator*, 2009

We now have all the mathematical machinery we need. This chapter explains the propagator model from first principles and makes precise why lattices are not merely convenient but *necessary*.

5.1 Cells: Accumulating Partial Information

The central insight of propagator networks is a shift in perspective:



A cell does not *store a value*; it stores *everything we know so far* about a value. New information can arrive from multiple sources and be merged in. The key requirement is that information can only *increase*: once we know something, we cannot un-know it.

This is precisely the structure of a *join-semilattice with a bottom element*: the bottom represents “we know nothing”, and merging two pieces of information is the join ($\sqcup$).

Definition — Information Order

A *bounded join-semilattice* is a partial order $(J, \leq)$ equipped with a join $\sqcup$ and a bottom $\perp$ such that:

- $a \sqcup b$ is the least upper bound of a and b ,
- $\perp \leq a$ for all a ,
- $a \leq b$ is read as “ b contains at least as much information as a ”.

We call this the *information order*.

```
1 namespace Propagators
2
```

```

3 class BoundedJoinSemilattice (α : Type) extends OrderBook.PartialOrder α where
4   sup : α → α → α
5   bot : α
6   le_sup_left  : ∀ (a b : α), a ≤ sup a b
7   le_sup_right : ∀ (a b : α), b ≤ sup a b
8   sup_le      : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
9   bot_le      : ∀ (a : α), bot ≤ a
10
11 -- Convenient notation
12 infixl:65 " ⊔ " => BoundedJoinSemilattice.sup
13 notation    "⊥"  => BoundedJoinSemilattice.bot
14
15 end Propagators

```

5.2 Why Information Can Only Increase

Suppose a propagator computes that “ x is between 3 and 7”. Later another propagator deduces that “ x is between 5 and 12”. Combining these we know “ x is between 5 and 7”. Notice: the combined knowledge is *strictly more specific* than either input.

If we allowed information to *decrease* (e.g., forgetting the $[3, 7]$ constraint and keeping only $[5, 12]$), we would lose soundness: our cell might claim values that the original propagators had ruled out.

Monotonicity Ensures Soundness

In a propagator network over a bounded join-semilattice, if every propagator is monotone, then the network is *sound*: the final cell values are consistent with all constraints.

Intuitively: since each step can only add information (move up in the lattice), the final state contains at least as much information as any intermediate state. Nothing is lost; nothing false is added (assuming correct propagators).

5.3 Propagators as Monotone Functions

Definition — Propagator

Given cells with values in $(J, \leq)$, a *propagator* is a monotone function $f : J^n \rightarrow J$ (reading from n input cells and writing to one output cell by merging the result).

Writing is done by merging: the output cell c receives $c \leftarrow c \sqcup f(\text{inputs})$. Since $\sqcup$ is the least upper bound, this can only increase c .

```

1 namespace Propagators
2
3 -- A unary propagator
4 structure Propagator1 (α β : Type)
5   [BoundedJoinSemilattice α]
6   [BoundedJoinSemilattice β] where
7   fn      : α → β
8   monotone : ∀ (a b : α), a ≤ b →
9     BoundedJoinSemilattice.sup (fn a) (fn b) = fn b
10
11 -- A binary propagator
12 structure Propagator2 (α β γ : Type)
13   [BoundedJoinSemilattice α]
14   [BoundedJoinSemilattice β]
15   [BoundedJoinSemilattice γ] where
16   fn      : α → β → γ
17   monotone : ∀ (a1 a2 : α) (b1 b2 : β),
18     a1 ≤ a2 → b1 ≤ b2 →
19     BoundedJoinSemilattice.sup (fn a1 b1) (fn a2 b2) = fn a2 b2
20
21 end Propagators

```

5.4 Networks, Scheduling, and Quiescence

A *propagator network* is a directed graph where nodes are cells and edges are propagators. Execution proceeds as follows:

1. Initialise all cells to $\perp$.
2. Place all propagators on a *work queue*.
3. While the queue is non-empty:
 - (a) Take a propagator p from the queue.
 - (b) Compute the new value $v = p(\text{input cells})$.
 - (c) Merge v into the output cell: $c \leftarrow c \sqcup v$.
 - (d) If c changed, add all propagators that read from c back to the queue.
4. The network has reached *quiescence* when the queue is empty.

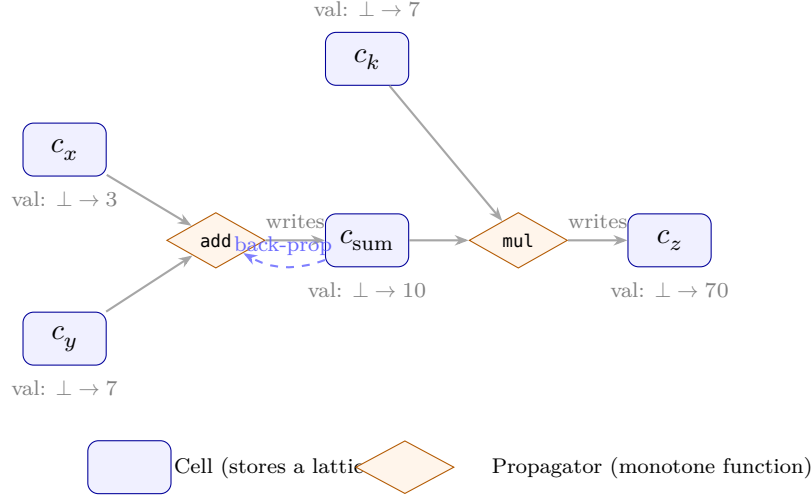


Figure 5.1: A small propagator network computing $x + y = \text{sum}$ and $\text{sum} \times k = z$ (with $x = 3$, $y = 7$, $k = 7$, so $\text{sum} = 10$ and $z = 70$). Cells (rounded rectangles) hold lattice values, initialised to $\perp$. Propagators (diamonds) read from their input cells and write to their output cell by merging with $\sqcup$. The dashed arrow shows back-propagation: when c_{sum} is updated the **add** propagator is re-queued to potentially narrow c_x or c_y .

Convergence

If the lattice satisfies the *ascending chain condition* (every strictly ascending sequence is finite), then the network reaches quiescence in finite time.

This is immediate: each cell value is non-decreasing and the chain condition guarantees it cannot increase indefinitely.

For our capstone projects, the domain lattice is always finite, so convergence is guaranteed.

5.5 The Fundamental Role of the Lattice

Let us be precise about why we need a lattice rather than merely some arbitrary type.

- **Join ($\sqcup$) for merging.** When two propagators independently derive information about the same cell, we need to combine their results. The join gives the *least informative* value that is *at least as informative* as both. No information is lost; no information is invented.
- **Bottom ($\perp$) for initialisation.** Before any propagator has run, a cell contains *no information*. The bottom element represents this “empty” state.
- **Partial order ($\leq$) for monitoring change.** We must know whether a cell’s value actually changed after merging, to decide whether to re-schedule dependent propagators. The partial order provides this: if $c \sqcup v = c$ (the new value adds no information), nothing changed.

- **Antisymmetry for termination.** If a cell's value can only go up and the lattice is finite, the antisymmetry of $\leq$ ensures each value is visited at most once.

Any type that fails to be a join-semilattice with $\perp$ is simply not a valid cell domain for a propagator network. The lattice structure is not a convenience — it is a prerequisite.

Chapter 6

Building a Propagator Network in Lean 4

Theory in hand, we now build a complete, runnable propagator network in Lean 4, starting from our typeclass definitions and ending with a working scheduler.

6.1 Mutable Cells

Lean 4 has an `IO` monad with mutable references (`IO.Ref`). We use these to implement cells.

```
1 namespace Propagators
2
3 -- A Cell wraps an IO.Ref and a type with a BoundedJoinSemilattice
4 structure Cell (α : Type) where
5   ref      : IO.Ref α
6
7 variable {α : Type} [BoundedJoinSemilattice α] [DecidableEq α]
8
9 -- Create a new cell initialised to ⊥
10 def Cell.new : IO (Cell α) := do
11   let r ← IO.mkRef (BoundedJoinSemilattice.bot (α := α))
12   return { ref := r }
13
14 -- Read the current value
15 def Cell.read (c : Cell α) : IO α := c.ref.get
16
17 -- Attempt to write new information.
18 -- Returns true if the cell's value increased.
19 def Cell.write (c : Cell α) (v : α) : IO Bool := do
20   let old ← c.ref.get
21   let merged := old ⊔ v
22   if merged == old then
23     return false -- no new information
24   else
25     c.ref.set merged
```

```

26     return true           -- cell updated, re-schedule dependents
27
28 end Propagators

```

We require `DecidableEq  $\alpha$` to compare `merged == old`. For the finite lattices we use in our capstone projects this is always available.

6.2 The Network and Scheduler

```

1 namespace Propagators
2
3 -- A propagator step is an IO action that returns
4 -- true if it caused any cell to change.
5 abbrev PropStep := IO Bool
6
7 -- Run a list of propagators to fixpoint.
8 -- Each pass re-runs all propagators; we stop when
9 -- a full pass causes no changes.
10 def runToFixpoint (steps : Array PropStep) : IO Unit := do
11   let mut changed := true
12   while changed do
13     changed := false
14     for step in steps do
15       let c ← step
16       if c then changed := true
17
18 -- A smarter scheduler that only re-runs propagators
19 -- whose inputs changed (using a dependency queue).
20 structure Network where
21   steps      : Array PropStep
22   -- In a production implementation, steps would be indexed
23   -- and cells would hold lists of dependent propagator IDs.
24   -- For clarity we use the simple "run all" scheduler here.
25
26 def Network.run (n : Network) : IO Unit :=
27   runToFixpoint n.steps
28
29 end Propagators

```

6.3 Building Propagators: Numeric Example

The simplest useful cell domain is the *flat lattice*: values are either completely unknown, exactly determined, or contradictory.

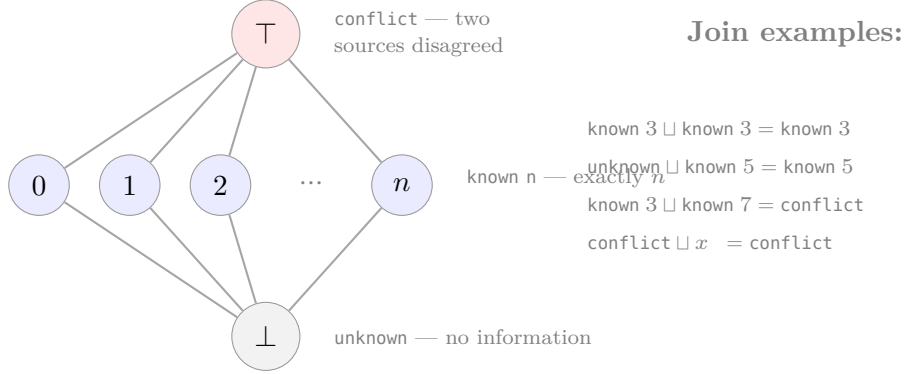


Figure 6.1: The flat natural number lattice **FlatNat**. The bottom $\perp$ represents “no information yet”. Each concrete value n is incomparable with all other concrete values — knowing the answer is 3 tells you nothing about whether it is 7. The top $\top$ represents a contradiction. This lattice has height 2, so any cell stabilises after at most two updates.

```

1 namespace Propagators
2
3 inductive FlatNat where
4   | unknown      : FlatNat      -- ⊥: no information
5   | known (n : Nat) : FlatNat    -- exactly n
6   | conflict     : FlatNat      -- ⊤: contradiction
7
8 instance : OrderBook.PartialOrder FlatNat where
9   le a b := match a, b with
10    | .unknown, _      => True      -- unknown ≤ everything
11    | _, .conflict     => True      -- everything ≤ conflict
12    | .known m, .known n => m = n  -- known values only ≤ themselves
13    | .conflict, .unknown => False
14    | .conflict, .known _ => False
15    | .known _, .unknown => False
16   le_refl := by intro a; cases a <=> simp
17   le_antisymm := by
18     intro a b hab hba
19     cases a <=> cases b <=> simp_all
20   le_trans := by
21     intro a b c hab hbc
22     cases a <=> cases b <=> cases c <=> simp_all
23
24 instance : BoundedJoinSemilattice FlatNat where

```

```

25   le      := OrderBook.PartialOrder.le
26   le_refl := OrderBook.PartialOrder.le_refl
27   le_antisymm := OrderBook.PartialOrder.le_antisymm
28   le_trans := OrderBook.PartialOrder.le_trans
29   bot      := .unknown
30   sup a b := match a, b with
31   | .unknown, x      => x
32   | x, .unknown      => x
33   | .known m, .known n => if m = n then .known m else .conflict
34   | _, _            => .conflict
35   bot_le      := by intro a; cases a <;> simp [OrderBook.PartialOrder.le]
36   sup_le_left := by intro a b; cases a <;> cases b <;> simp_all
    ↪ [OrderBook.PartialOrder.le]
37   sup_le_right := by intro a b; cases a <;> cases b <;> simp_all
    ↪ [OrderBook.PartialOrder.le]
38   le_sup      := by
39     intro a b c hac hbc
40     cases a <;> cases b <;> cases c <;> simp_all [OrderBook.PartialOrder.le]
41
42   deriving instance DecidableEq for FlatNat
43
44   end Propagators

```

6.4 A Complete Running Example

Example — Arithmetic propagation

We know: $x + y = 10$ and $x = 3$. A propagator should deduce $y = 7$.

```

1   namespace Propagators
2
3   open FlatNat
4
5   -- Addition propagator: given x and y cells, write x+y to sum cell
6   def addProp (cx cy csum : Cell FlatNat) : PropStep := do
7     let x ← cx.read
8     let y ← cy.read
9     match x, y with
10    | .known a, .known b => csum.write (.known (a + b))
11    | _, _ => return false
12
13   -- Subtraction (used to deduce y = sum - x)
14   def subProp (csum cx cy : Cell FlatNat) : PropStep := do

```

```

15   let s ← csum.read
16   let x ← cx.read
17   match s, x with
18   | .known total, .known a =>
19     if total ≥ a
20     then cy.write (.known (total - a))
21     else cy.write .conflict
22   | _, _ => return false
23
24 def exampleNetwork : IO Unit := do
25   let cx ← Cell.new (α := FlatNat)
26   let cy ← Cell.new (α := FlatNat)
27   let csum ← Cell.new (α := FlatNat)
28
29   -- Seed known facts
30   let _ ← cx.write (.known 3)
31   let _ ← csum.write (.known 10)
32
33   let net : Network := {
34     steps := #[
35       addProp cx cy csum,
36       subProp csum cx cy,
37       subProp csum cy cx
38     ]
39   }
40
41   net.run
42
43   let yVal ← cy.read
44   match yVal with
45   | .known n => IO.println s!"y = {n}"    -- prints: y = 7
46   | .unknown => IO.println "y is unknown"
47   | .conflict => IO.println "contradiction!"
48
49 end Propagators

```

6.5 Why the Lattice Is Load-Bearing Here

Notice several things:

- `Cell.write` can be called multiple times from different propagators without coordination. The join ensures the final value is consistent with all of them.
- Running `addProp` when either input is `unknown` correctly does nothing (returns `false`), waiting for more information.

- The `conflict` element propagates correctly: if two propagators derive incompatible values, the join produces `conflict`, which then propagates to any cell that depends on this one.
- The whole system terminates because `FlatNat` has height 2: after at most two increases, any cell stabilises.

6.6 Exercises

Exercise 6.1 — Three-cell constraint

Add a `mulProp` propagator for $x \times y = z$ with the `FlatNat` lattice. Build a network with $x \times y = 12$, $x = 3$, and verify it deduces $y = 4$.

Exercise 6.2 — Conflict detection

Seed a network with $x = 3$ from one source and $x = 5$ from another. Verify that `cx.read` returns `.conflict`.

Chapter 7

The Interval Lattice

Flat lattices are powerful but coarse: a cell either knows the exact value or knows nothing. *Interval lattices* let us represent partial numeric knowledge as a range, enabling *bound propagation* — the engine behind many real-world constraint solvers.

7.1 Intervals as Partial Information

An interval $[l, h]$ says: “the true value lies between l and h ”. The wider the interval, the less we know. The narrowest possible interval is a single point $[n, n]$, meaning we know the value exactly.

Definition — Interval Lattice

Let the domain be extended integers $\mathbb{Z}_\infty = \mathbb{Z} \cup \{-\infty, +\infty\}$. Define:

- An *interval* is either $\perp$ (no values possible, contradiction) or $[l, h]$ with $l \leq h$.
- The *information order*: $[l_1, h_1] \leq [l_2, h_2] \iff l_2 \leq l_1 \wedge h_1 \leq h_2$ (a narrower interval contains more information).
- $\perp \leq [l, h]$ for all intervals.
- The *join* (merge two pieces of information): $[l_1, h_1] \sqcup [l_2, h_2] = [\min(l_1, l_2), \max(h_1, h_2)]$ (the smallest interval containing both).
- $\perp$ is the identity for $\sqcup$.

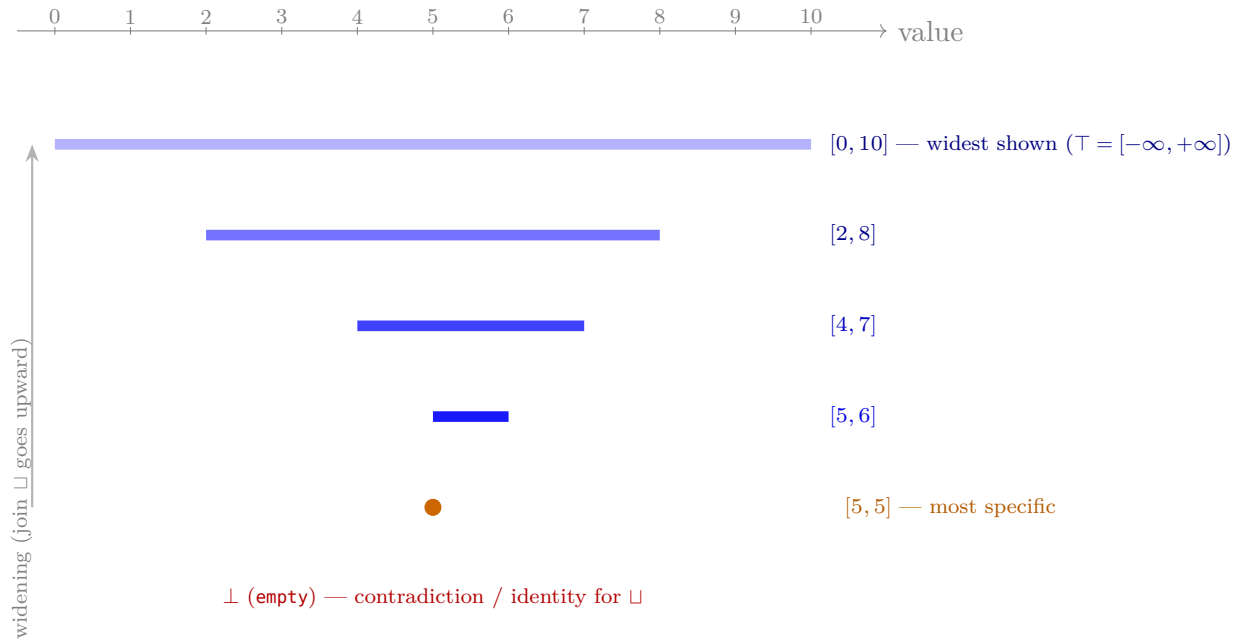


Figure 7.1: The interval lattice order as used in this book’s propagators. $\perp$ (empty) sits at the bottom as the join identity: $\perp \sqcup x = x$. Going upward, intervals widen. The join (`Interval.sup`) of two ranges is their *hull* (smallest interval containing both), which is at least as wide as either input and therefore weakly higher in the order. The meet (`Interval.intersect`) is the intersection, which is narrower and thus lower. The widest possible interval $[-\infty, +\infty]$ is $\top$.

```

1 namespace Intervals
2
3 inductive Interval where
4   | empty          : Interval --  $\perp$ 
5   | range (lo hi : Int) (h : lo ≤ hi) : Interval
6
7 -- Information order: narrower = more informative = higher
8 def Interval.le (a b : Interval) : Prop :=
9   match a, b with
10  | .empty, _      => True
11  | .range _ _ _, .empty => False
12  | .range l1 h1 _, .range l2 h2 _ => l2 ≤ l1 ∧ h1 ≤ h2
13
14 -- Join: widen to contain both intervals
15 def Interval.sup (a b : Interval) : Interval :=
16   match a, b with
17   | .empty, x => x
18   | x, .empty => x
19   | .range l1 h1 _, .range l2 h2 _ =>
20     let l := min l1 l2
21     let h := max h1 h2

```



```

22     .range l h (by omega)
23
24 instance : OrderBook.PartialOrder Interval where
25   le      := Interval.le
26   le_refl := by intro a; cases a <=> simp [Interval.le]
27   le_antisymm := by
28     intro a b hab hba
29     cases a <=> cases b <=>
30       simp [Interval.le] at * <=> omega
31   le_trans := by
32     intro a b c hab hbc
33     cases a <=> cases b <=> cases c <=>
34       simp [Interval.le] at * <=> omega
35
36 end Intervals

```

7.2 Interval Arithmetic Propagators

With the interval lattice, we can implement real *bound propagation*.

```

1 namespace Intervals
2
3 -- Add two intervals
4 def Interval.add (a b : Interval) : Interval :=
5   match a, b with
6   | .empty, _ | _, .empty => .empty
7   | .range l1 h1 _, .range l2 h2 _ =>
8     .range (l1 + l2) (h1 + h2) (by omega)
9
10 -- Subtract: a - b = [l1-h2, h1-l2]
11 def Interval.sub (a b : Interval) : Interval :=
12   match a, b with
13   | .empty, _ | _, .empty => .empty
14   | .range l1 h1 _, .range l2 h2 _ =>
15     .range (l1 - h2) (h1 - l2) (by omega)
16
17 -- Intersect (meet in the information order – more precise)
18 def Interval.intersect (a b : Interval) : Interval :=
19   match a, b with
20   | .empty, _ | _, .empty => .empty
21   | .range l1 h1 _, .range l2 h2 _ =>
22     let l := max l1 l2
23     let h := min h1 h2
24     if h_le : l ≤ h then .range l h h_le else .empty

```

```

25
26 end Intervals

```

7.3 A Worked Example: Bound Propagation

Example — $x + y = 10, x \in [2, 8]$

What is the tightest interval for y ?

From $x + y = 10$ we get $y = 10 - x$. Since $x \in [2, 8]$, we have $y \in [10 - 8, 10 - 2] = [2, 8]$. The propagator should derive this automatically.

```

1 namespace Intervals
2
3 -- Propagator: sum = x + y (bidirectional: also narrows x and y)
4 def addConstraint
5   (cx cy csum : Propagators.Cell Interval) : Array Propagators.PropStep :=
6   #[
7     -- Forward: sum ← x + y
8     do
9       let x ← cx.read; let y ← cy.read
10      csum.write (Interval.add x y),
11     -- Backward: x ← sum - y
12     do
13       let s ← csum.read; let y ← cy.read
14       cx.write (Interval.sub s y),
15     -- Backward: y ← sum - x
16     do
17       let s ← csum.read; let x ← cx.read
18       cy.write (Interval.sub s x)
19   ]
20
21 def intervalExample : IO Unit := do
22   let cx ← Propagators.Cell.new (α := Interval)
23   let cy ← Propagators.Cell.new (α := Interval)
24   let csum ← Propagators.Cell.new (α := Interval)
25
26   -- x ∈ [2, 8], sum = 10
27   let _ ← cx.write (.range 2 8 (by omega))
28   let _ ← csum.write (.range 10 10 (by omega))
29
30   let steps := addConstraint cx cy csum
31   Propagators.runToFixpoint steps
32

```

```

33   let yVal ← cy.read
34   match yVal with
35   | .empty      => IO.println "y: impossible"
36   | .range l h _ => IO.println s!"y ∈ [{l}, {h}]"  -- y ∈ [2, 8]
37
38 end Intervals

```

The join in action. When the backward propagator writes $[2, 8]$ to c_y and c_y already holds $\perp$, the join $\perp \sqcup [2, 8] = [2, 8]$ correctly updates the cell. On the next pass, the forward propagator reads $x = [2, 8]$ and $y = [2, 8]$, computes $\text{add} = [4, 16]$, and writes to c_{sum} . Since $[10, 10] \sqcup [4, 16] = [4, 16]$ would *widen* the constraint (lose information), we could use `intersect` here for a more powerful solver. We leave this as Exercise 7.1.

7.4 Exercises

Exercise 7.1 — Intersection for tighter bounds

Replace `Interval.sup` (widening join) with `Interval.intersect` (narrowing meet) in the cell write operation for the sum constraint. Explain why the information order must be *reversed* when you switch from widening to narrowing. What does this say about the dual lattice?

Exercise 7.2 — Multiplication intervals

Implement `Interval.mul`. Be careful: multiplying negative intervals requires considering all four corner products.

Part III

Capstone Projects

Chapter 8

Capstone I: A Sudoku Solver

Sudoku is a constraint satisfaction problem: each of 81 cells must receive a digit 1–9 subject to row, column, and box uniqueness constraints. Propagator networks handle this beautifully. In this chapter we build a complete solver using nothing but lattice-ordered propagation.

8.1 The Domain Lattice

Each Sudoku cell can be modelled as a set of *candidate digits*. Initially every cell can hold any of $\{1, \dots, 9\}$. Constraints eliminate candidates.

Definition — Candidate Set Lattice

Let $D = \{1, \dots, 9\}$. The *candidate-set lattice* is the powerset $\mathcal{P}(D)$ ordered by *reverse inclusion*:

$$S \leq T \iff T \subseteq S.$$

The bottom $\perp = D$ (no information: any digit possible), the top $\top = \emptyset$ (contradiction: no digit possible). The join is intersection: $S \sqcup T = S \cap T$ (the smallest set of candidates consistent with both sources).

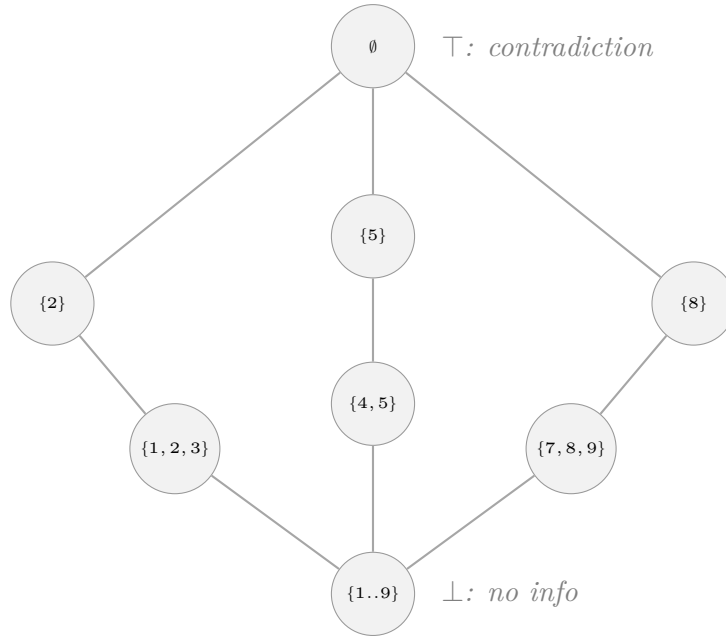


Figure 8.1: A fragment of the candidate-set lattice (reverse inclusion order).

This is precisely the *dual* powerset lattice from Exercise 1.4!

```

1 namespace Sudoku
2
3 -- Digits 1..9, represented as Fin 9 (0..8 internally, displayed as 1..9)
4 abbrev Digit := Fin 9
5
6 -- A candidate set: which digits are still possible for this cell
7 -- We represent it as a Bool array of length 9
8 structure CandidateSet where
9   present : Fin 9 → Bool
10
11 def CandidateSet.full : CandidateSet :=
12   { present := fun _ => true }
13
14 def CandidateSet.empty : CandidateSet :=
15   { present := fun _ => false }
16
17 def CandidateSet.singleton (d : Digit) : CandidateSet :=
18   { present := fun i => i == d }
19
20 def CandidateSet.remove (s : CandidateSet) (d : Digit) : CandidateSet :=
21   { present := fun i => s.present i && !(i == d) }
22
23 def CandidateSet.inter (s t : CandidateSet) : CandidateSet :=
24   { present := fun i => s.present i && t.present i }
25

```

```

26 def CandidateSet.union (s t : CandidateSet) : CandidateSet :=
27   { present := fun i => s.present i || t.present i }
28
29 -- The information order:  $s \leq t$  iff  $t \subseteq s$  ( $t$  is more specific)
30 def CandidateSet.le (s t : CandidateSet) : Prop :=
31    $\forall i : \text{Digit}, t.\text{present } i = \text{true} \rightarrow s.\text{present } i = \text{true}$ 
32
33 instance : OrderBook.PartialOrder CandidateSet where
34   le := CandidateSet.le
35   le_refl := fun s i h => h
36   le_antisymm := fun s t h1 h2 =>
37     show s = t from by
38       cases s; cases t
39       congr 1; funext i
40       cases hi : t.present i
41       · simp [CandidateSet.le] at h1
42       cases hs : s.present i
43       · rfl
44       · exact absurd (h1 i hs) (by simp [hi])
45       · exact h2 i hi
46   le_trans := fun s t u hst htui hi => hst i (htui i hi)
47
48 -- Join = intersection (combines constraints)
49 instance : Propagators.BoundedJoinSemilattice CandidateSet where
50   le := CandidateSet.le
51   le_refl := fun s i h => h
52   le_antisymm := OrderBook.PartialOrder.le_antisymm ( $\alpha := \text{CandidateSet}$ )
53   le_trans := OrderBook.PartialOrder.le_trans ( $\alpha := \text{CandidateSet}$ )
54   bot := CandidateSet.full
55   sup := CandidateSet.inter --  $\sqcup = \cap$  in information order!
56   bot_le := fun s i _ => rfl -- full  $\leq s$  since  $s \subseteq \text{full}$ 
57   sup_le_left := fun s t i h => (Bool.and_eq_true.mp h).1
58   sup_le_right := fun s t i h => (Bool.and_eq_true.mp h).2
59   le_sup := fun s t u hsu htui h =>
60     Bool.and_eq_true.mpr (hsu i h, htui i h)
61
62 instance : DecidableEq CandidateSet :=
63   fun s t =>
64     if h :  $\forall i, s.\text{present } i = t.\text{present } i$ 
65     then isTrue (by cases s; cases t; congr 1; funext i; exact h i)
66     else isFalse (fun heq => h (fun i => by rw [heq]))
67
68 end Sudoku

```

8.2 Representing the Grid

```
1 namespace Sudoku
2
3 -- 81 cells indexed by row and column (both Fin 9)
4 abbrev Grid := Fin 9 → Fin 9 → Propagators.Cell CandidateSet
5
6 def makeGrid : IO Grid := do
7   let mut rows : Array (Array (Propagators.Cell CandidateSet)) := #[]
8   for _ in [:9] do
9     let mut row : Array (Propagators.Cell CandidateSet) := #[]
10    for _ in [:9] do
11      let c ← Propagators.Cell.new (α := CandidateSet)
12      row := row.push c
13    rows := rows.push row
14  -- Convert to function (safe because sizes match)
15  return fun r c => rows[r.val]![c.val]!
16
17 -- Seed a given digit into a cell
18 def seedCell (grid : Grid) (r c : Fin 9) (d : Digit) : IO Unit := do
19   let _ ← (grid r c).write (CandidateSet.singleton d)
20
21 end Sudoku
```

8.3 Constraint Propagators

Sudoku has three families of constraints: row, column, and 3×3 box uniqueness. Each is expressed as a propagator that, given a cell known to hold digit d , removes d from all peers.

```
1 namespace Sudoku
2
3 -- If cell (r,c) is determined to be d, remove d from all other cells in row r
4 def rowPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
5   let cands ← (grid r c).read
6   -- Check if this cell is a singleton
7   let det : Option Digit := Id.run do
8     let mut found : Option Digit := none
9     let mut count := 0
10    for i in [:9] do
11      let i' : Digit := ⟨i, by omega⟩
12      if cands.present i' then
13        found := some i'; count := count + 1
```



```

14   if count = 1 then found else none
15 match det with
16 | none => return false
17 | some d =>
18   let mut changed := false
19   for j in [:9] do
20     let j' : Fin 9 := ⟨j, by omega⟩
21     if j' ≠ c then
22       let removed := { present := fun i => (grid r j').read.present i && !(i
23         ↪ == d) }
24       -- We re-read and merge
25       let old ← (grid r j').read
26       let upd := CandidateSet.inter old { present := fun i => !(i == d) }
27       let c ← (grid r j').write upd
28       if c then changed := true
29   return changed
30
31 -- Analogous: column propagator
32 def colPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
33   let cands ← (grid r c).read
34   let det : Option Digit := Id.run do
35     let mut found : Option Digit := none
36     let mut count := 0
37     for i in [:9] do
38       let i' : Digit := ⟨i, by omega⟩
39       if cands.present i' then found := some i'; count := count + 1
40     if count = 1 then found else none
41 match det with
42 | none => return false
43 | some d =>
44   let mut changed := false
45   for i in [:9] do
46     let i' : Fin 9 := ⟨i, by omega⟩
47     if i' ≠ r then
48       let old ← (grid i' c).read
49       let upd := CandidateSet.inter old { present := fun k => !(k == d) }
50       let ch ← (grid i' c).write upd
51       if ch then changed := true
52   return changed
53
54 -- Box propagator (which 3×3 box contains (r,c)?)
55 def boxPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
56   let cands ← (grid r c).read
57   let det : Option Digit := Id.run do
58     let mut found : Option Digit := none; let mut count := 0
59     for i in [:9] do

```

```
59     let i' : Digit := (i, by omega)
60     if cands.present i' then found := some i'; count := count + 1
61     if count = 1 then found else none
62 match det with
63 | none => return false
64 | some d =>
65     let boxR := r.val / 3 * 3
66     let boxC := c.val / 3 * 3
67     let mut changed := false
68     for dr in [:3] do
69         for dc in [:3] do
70             let r' : Fin 9 := (boxR + dr, by omega)
71             let c' : Fin 9 := (boxC + dc, by omega)
72             if !(r' = r && c' = c) then
73                 let old ← (grid r' c').read
74                 let upd := CandidateSet.inter old { present := fun k => !(k == d) }
75                 let ch ← (grid r' c').write upd
76                 if ch then changed := true
77     return changed
78
79 end Sudoku
```

8.4 Assembling and Running the Solver

```
1 namespace Sudoku
2
3 def buildNetwork (grid : Grid) : Array Propagators.PropStep := Id.run do
4     let mut steps : Array Propagators.PropStep := #[]
5     for r in [:9] do
6         for c in [:9] do
7             let r' : Fin 9 := (r, by omega)
8             let c' : Fin 9 := (c, by omega)
9             steps := steps.push (rowPropagator grid r' c')
10            steps := steps.push (colPropagator grid r' c')
11            steps := steps.push (boxPropagator grid r' c')
12    return steps
13
14 -- Print the grid (showing the unique digit or '?' for unsolved cells)
15 def printGrid (grid : Grid) : IO Unit := do
16     for r in [:9] do
17         let mut row := ""
18         for c in [:9] do
19             let r' : Fin 9 := (r, by omega)
```

```

20     let c' : Fin 9 := (c, by omega)
21     let cands ← (grid r' c').read
22     let mut digit := '?'
23     let mut count := 0
24     for i in [:9] do
25         let i' : Digit := (i, by omega)
26         if cands.present i' then digit := Char.ofNat (i + 49); count := count
27             ↪ + 1
28         row := row ++ (if count = 1 then s! "{digit}" else "?")
29         if c % 3 == 2 && c < 8 then row := row ++ "|"
30     IO.println row
31     if r % 3 == 2 && r < 8 then IO.println "-----"
32
33 -- Example: a simple Sudoku puzzle
34 -- (0 = unknown, digits are given clues)
35 def examplePuzzle : Array (Array Nat) := #[
36     #[5, 3, 0, 0, 7, 0, 0, 0, 0],
37     #[6, 0, 0, 1, 9, 5, 0, 0, 0],
38     #[0, 9, 8, 0, 0, 0, 0, 6, 0],
39     #[8, 0, 0, 0, 6, 0, 0, 0, 3],
40     #[4, 0, 0, 8, 0, 3, 0, 0, 1],
41     #[7, 0, 0, 0, 2, 0, 0, 0, 6],
42     #[0, 6, 0, 0, 0, 0, 2, 8, 0],
43     #[0, 0, 0, 4, 1, 9, 0, 0, 5],
44     #[0, 0, 0, 0, 8, 0, 0, 7, 9]
45 ]
46
47 def solveSudoku : IO Unit := do
48     let grid ← makeGrid
49     -- Seed known digits
50     for r in [:9] do
51         for c in [:9] do
52             let d := examplePuzzle[r][c]!
53             if d ≠ 0 then
54                 let r' : Fin 9 := (r, by omega)
55                 let c' : Fin 9 := (c, by omega)
56                 seedCell grid r' c' (d - 1, by omega)
57
58 -- Build and run the propagator network
59 let steps := buildNetwork grid
60 Propagators.runToFixpoint steps
61 -- Print the result
62 IO.println "Solution:"
63 printGrid grid

```

8.5 Why This Works: The Lattice Perspective

- Each cell starts at $\perp = \{1, \dots, 9\}$: no information.
- Every propagator only writes using `CandidateSet.inter` (the join in the information order), which can only *reduce* candidate sets — i.e., move *up* in the information order.
- The network terminates because the candidate-set lattice is finite and the cells only move upward.
- Reaching $\top = \emptyset$ in any cell signals a contradiction.
- A solved cell is one that has reached a singleton $\{d\}$.

The elegant insight: *solving Sudoku is computing the least fixed point of the combined propagation function*, where the partial order is the information order on candidate sets. Knaster–Tarski guarantees it exists; our scheduler computes it.

Exercise 8.1 — Naked pair

A *naked pair* occurs when two cells in the same unit both have exactly the same two candidates $\{a, b\}$. Then a and b can be removed from all other cells in that unit. Implement this as an additional propagator.

Exercise 8.2 — Contradiction detection

Add a `contradictionProp` that checks every cell; if any cell reaches $\top = \emptyset$, print a diagnostic and halt. Why is this detection sound (i.e., why can't a cell reach $\emptyset$ unless the puzzle is truly unsatisfiable)?

Chapter 9

Capstone II: Type Inference via Propagation

Our second capstone takes us into programming language theory. Type inference — the process of automatically deducing types of expressions — is naturally formulated as constraint propagation on a type lattice.

9.1 A Simple Expression Language

We work with a fragment of the lambda calculus extended with integers and booleans.

```
1 namespace TypeInfer
2
3 -- Expressions
4 inductive Expr where
5   | intLit (n : Int)      : Expr
6   | boolLit (b : Bool)   : Expr
7   | var     (x : String)  : Expr
8   | add     (e1 e2 : Expr) : Expr
9   | ifExpr  (cond t f : Expr) : Expr
10  | lam     (x : String) (body : Expr) : Expr
11  | app     (fn arg : Expr) : Expr
12  | letExpr (x : String) (e body : Expr) : Expr
13
14 end TypeInfer
```

9.2 The Type Lattice

Rather than types being a simple set, we arrange them in a lattice that captures the degree to which a type is *known*.

Definition — Flat Type Lattice

Define the type information domain:

- $\perp$ (Unknown): no type information yet.
- TInt , TBool : concrete base types.
- $\text{TFun } s \ t$: function type, parameterised by the type information for the domain and codomain (which are themselves in the lattice).
- $\top$ (TypeError): contradiction (incompatible constraints).

The order is: $\perp \leq$ everything, everything $\leq \top$, and base types are only $\leq$ themselves (and $\perp/\top$). Function types:

$\text{TFun } s_1 \ t_1 \leq \text{TFun } s_2 \ t_2$ iff $s_1 \leq s_2 \wedge t_1 \leq t_2$.

```

1 namespace TypeInfer
2
3 inductive Ty where
4   | unknown      : Ty    --  $\perp$ 
5   | int          : Ty
6   | bool         : Ty
7   | fun_ (dom cod : Ty) : Ty    --  $dom \rightarrow cod$ 
8   | typeError    : Ty    --  $\top$ 
9
10 -- Unification of two types: compute their join
11 def Ty.unify : Ty → Ty → Ty
12   | .unknown, t      => t
13   | t, .unknown      => t
14   | .typeError, _    => .typeError
15   | _, .typeError    => .typeError
16   | .int, .int       => .int
17   | .bool, .bool     => .bool
18   | .fun_ d1 c1, .fun_ d2 c2 => .fun_ (Ty.unify d1 d2) (Ty.unify c1 c2)
19   | _, _            => .typeError -- incompatible types
20
21 -- The information order
22 def Ty.le : Ty → Ty → Prop
23   | .unknown, _      => True
24   | _, .typeError    => True
25   | .int, .int       => True
26   | .bool, .bool     => True
27   | .fun_ d1 c1, .fun_ d2 c2 => Ty.le d1 d2 ∧ Ty.le c1 c2
28   | _, _            => False
29
30 instance : OrderBook.PartialOrder Ty where
31   le      := Ty.le
32   le_refl := by intro t; induction t <;> simp [Ty.le, *]
33   le_antisymm := by
34     intro t1 t2 h12 h21

```

```

35   induction t1 <=> cases t2 <=> simp [Ty.le] at * <=>
36     try (first | rfl | exact absurd h12 id | exact absurd h21 id)
37     case fun_.fun_ d1 c1 ih_d ih_c d2 c2 =>
38       have := ih_d h12.1 h21.1; have := ih_c h12.2 h21.2; simp_all
39   le_trans      := by
40     intro t1 t2 t3 h12 h23
41     induction t1 <=> cases t2 <=> cases t3 <=>
42       simp [Ty.le] at * <=> try tauto
43     case fun_.fun_.fun_ => exact (by tauto, by tauto)
44
45 instance : Propagators.BoundedJoinSemilattice Ty where
46   le      := Ty.le
47   le_refl := OrderBook.PartialOrder.le_refl
48   le_antisymm := OrderBook.PartialOrder.le_antisymm
49   le_trans := OrderBook.PartialOrder.le_trans
50   bot      := .unknown
51   sup      := Ty.unify
52   bot_le   := fun _ => trivial
53   sup_le_left := by
54     intro a b; induction a <=> cases b <=>
55       simp [Ty.le, Ty.unify, *] <=> try exact (by assumption, by assumption)
56   sup_le_right := by
57     intro a b; induction b <=> cases a <=>
58       simp [Ty.le, Ty.unify, *] <=> try exact (by assumption, by assumption)
59   le_sup      := by
60     intro a b c hac hbc
61     induction a <=> cases b <=> cases c <=>
62       simp [Ty.le, Ty.unify] at * <=> tauto
63
64 deriving instance DecidableEq for Ty
65
66 end TypeInfer

```

9.3 Type Constraint Generation

We generate constraints by walking the expression tree. Each sub-expression gets a fresh *type cell*. Constraints are propagators that merge type information into these cells.

```

1 namespace TypeInfer
2
3 -- A typing context maps variable names to type cells
4 abbrev Context := List (String × Propagators.Cell Ty)
5

```

```
6 def Context.lookup (ctx : Context) (x : String) :
7   Option (Propagators.Cell Ty) :=
8   ctx.find? (fun p => p.1 == x) |>.map (·.2)
9
10 -- Generate type cells and propagators for an expression
11 -- Returns the type cell for this expression.
12 partial def inferExpr
13   (ctx : Context)
14   (e : Expr)
15   (steps : IO.Ref (Array Propagators.PropStep)) :
16   IO (Propagators.Cell Ty) := do
17   match e with
18   | .intLit _ =>
19     let c ← Propagators.Cell.new (α := Ty)
20     let _ ← c.write .int
21     return c
22
23   | .boolLit _ =>
24     let c ← Propagators.Cell.new (α := Ty)
25     let _ ← c.write .bool
26     return c
27
28   | .var x =>
29     match ctx.lookup x with
30     | some c => return c
31     | none   =>
32       let c ← Propagators.Cell.new (α := Ty)
33       let _ ← c.write .typeError -- unbound variable
34       return c
35
36   | .add e1 e2 =>
37     let c1 ← inferExpr ctx e1 steps
38     let c2 ← inferExpr ctx e2 steps
39     let cOut ← Propagators.Cell.new (α := Ty)
40     -- Propagator: both inputs and output must be Int
41     steps.modify (· .push do
42       let _ ← c1.write .int
43       let _ ← c2.write .int
44       cOut.write .int)
45     return cOut
46
47   | .ifExpr cond t f =>
48     let ccond ← inferExpr ctx cond steps
49     let ct    ← inferExpr ctx t     steps
50     let cf    ← inferExpr ctx f     steps
51     let cOut  ← Propagators.Cell.new (α := Ty)
```



```

52   -- Propagator: cond must be Bool; t and f must agree; out = t
53   steps.modify (· .push do
54     let _ ← ccond.write .bool
55     let tv ← ct.read; let fv ← cf.read
56     let _ ← ct.write fv; let _ ← cf.write tv
57     let tv' ← ct.read
58     cOut.write tv')
59   return cOut
60
61 | .lam x body =>
62   let cDom ← Propagators.Cell.new (α := Ty)
63   let ctx' := (x, cDom) :: ctx
64   let cBody ← inferExpr ctx' body steps
65   let cOut ← Propagators.Cell.new (α := Ty)
66   -- Propagator: out = dom → body
67   steps.modify (· .push do
68     let d ← cDom.read; let b ← cBody.read
69     cOut.write (.fun_ d b))
70   return cOut
71
72 | .app fn arg =>
73   let cFn ← inferExpr ctx fn steps
74   let cArg ← inferExpr ctx arg steps
75   let cOut ← Propagators.Cell.new (α := Ty)
76   -- Propagator: fn must be (arg → out)
77   steps.modify (· .push do
78     let fv ← cFn.read; let av ← cArg.read
79     let ov ← cOut.read
80     match fv with
81     | .fun_ d r =>
82       let _ ← cArg.write d -- arg type must match domain
83       let _ ← cOut.write r -- result type from codomain
84       let _ ← cFn.write (.fun_ av ov) -- refine fn type
85       return false -- (actual change tracked by writes)
86     | .unknown =>
87       let _ ← cFn.write (.fun_ av ov)
88       return false
89     | _ =>
90       let _ ← cFn.write .typeError; return false)
91   return cOut
92
93 | .letExpr x e body =>
94   let ce ← inferExpr ctx e steps
95   let ctx' := (x, ce) :: ctx
96   inferExpr ctx' body steps
97

```

```
98 end TypeInfer
```

9.4 Running the Type Inferencer

```
1 namespace TypeInfer
2
3 def infer (e : Expr) : IO Ty := do
4   let stepsRef ← IO.mkRef (Array.empty : Array Propagators.PropStep)
5   let cResult  ← inferExpr [] e stepsRef
6   let steps    ← stepsRef.get
7   Propagators.runToFixpoint steps
8   cResult.read
9
10  -- Helper to show types
11  def Ty.toString : Ty → String
12    | .unknown      => "?"
13    | .int          => "Int"
14    | .bool         => "Bool"
15    | .fun_d r      => s! "{Ty.toString d} → {Ty.toString r}"
16    | .typeError    => "TypeError"
17
18  -- Examples
19  def ex1 : Expr := .add (.intLit 1) (.intLit 2)
20  -- Expected: Int
21
22  def ex2 : Expr := .lam "x" (.add (.var "x") (.intLit 1))
23  -- Expected: (Int → Int)
24
25  def ex3 : Expr := .ifExpr (.boolLit true) (.intLit 1) (.intLit 2)
26  -- Expected: Int
27
28  def ex4 : Expr := .add (.boolLit true) (.intLit 1)
29  -- Expected: TypeError (Bool ≠ Int)
30
31  def runExamples : IO Unit := do
32    for (name, e) in [("1 + 2", ex1), ("λx. x+1", ex2),
33                     ("if true 1 2", ex3), ("true + 1", ex4)] do
34      let ty ← infer e
35      IO.println s! "{name} : {Ty.toString ty}"
36
37  end TypeInfer
```

Expected output:

```
1 + 2 : Int
λx. x+1 : (Int → Int)
if true 1 2 : Int
true + 1 : TypeError
```

9.5 What the Lattice Buys Us Here

- **Bidirectional propagation is free.** When we call `cArg.write d` inside the `app` propagator, we are propagating type information *backwards*: from the function type to the argument. The lattice join ensures this never loses previously-inferred information.
- **Inconsistency is explicit.** `TypeError` is the $\top$ element. When two incompatible type constraints are merged (`Ty.unify .int .bool = .typeError`), the cell value jumps to $\top$ and stays there. No exception is thrown; the type error propagates naturally through the lattice.
- **Incrementality is free.** If we learn a new fact about x 's type (say, x appears in another expression), we just write to x 's type cell and re-run the network. The lattice join handles merging old and new constraints correctly.
- **Ordering captures “how much we know”.** $\perp = \text{Unknown} < \text{Int} < \text{TypeError} = \top$. The height of a cell's value in the lattice is a measure of how constrained it is. A fully-typed program has all cells at concrete types (neither $\perp$ nor $\top$).

9.6 Exercises

Exercise 9.1 — Let-polymorphism

Lean's type system supports polymorphism. Extend `Ty` with a `TVar (name : String)` constructor and modify `Ty.unify` to do proper unification (substituting type variables). How does this affect the lattice structure?

Exercise 9.2 — Recursive types

Add a `letRec` construct to `Expr` and extend the inferencer to handle it. Note: the type cell for the recursive binding must be initialised to `Unknown` and refined by propagation, enabling inference of e.g. `let rec f x = if x = 0 then 1 else x * f (x-1) .`

Exercise 9.3 — Error messages

When a cell reaches `TypeError`, record *why* (which two types conflicted). Augment the `Ty` type so that `TypeError` carries a `String` explanation, and thread error messages through `Ty.unify`.

Chapter 10

Lean 4 Quick Reference

10.1 Basic Types and Syntax

```
1  -- Type universes
2  #check Nat      -- Nat : Type
3  #check Type     -- Type : Type 1
4  #check Prop     -- Prop : Sort 0
5
6  -- Function types
7  def f : Nat → Nat → Nat := fun a b => a + b
8  def g (a b : Nat) : Nat := a + b    -- same, different syntax
9
10 -- Dependent types
11 def myList (α : Type) (n : Nat) : Type := ...
12
13 -- Let bindings
14 def h : Nat :=
15   let x := 5
16   let y := x + 1
17   x * y
```

10.2 Typeclasses

```
1  -- Declaring a typeclass
2  class MyClass (α : Type) where
3    someMethod : α → Nat
4    someProperty : ∀ (a : α), someMethod a ≥ 0
5
6  -- Providing an instance
7  instance : MyClass Nat where
8    someMethod := id
9    someProperty := Nat.zero_le
10
```

```
11 -- Using an instance
12 def useMyClass [MyClass  $\alpha$ ] (a :  $\alpha$ ) : Nat :=
13   MyClass.someMethod a
```

10.3 Tactics Reference

Tactic	Usage
<code>rfl</code>	Prove $a = a$ by reflexivity
<code>exact h</code>	Close goal with proof term <code>h</code>
<code>apply f</code>	Apply function/lemma <code>f</code>
<code>intro h</code>	Introduce a hypothesis
<code>cases h</code>	Case split on <code>h</code>
<code>induction n</code>	Induction on <code>n</code>
<code>simp</code>	Simplify using simp lemmas
<code>simp_all</code>	Simplify using all hypotheses
<code>omega</code>	Linear arithmetic over $\mathbb{Z}$
<code>decide</code>	Decide propositions by computation
<code>constructor</code>	Introduce an And/Iff/...
<code>have h := ...</code>	Introduce a local lemma
<code>tauto</code>	Tautology checker
<code>funext</code>	Function extensionality
<code>congr</code>	Congruence

Table 10.1: Commonly used Lean 4 tactics in this book.

10.4 Unicode Input

Symbol	Input	Meaning
$\leq$	<code>\le</code>	Less-than-or-equal
$\geq$	<code>\ge</code>	Greater-than-or-equal
$\rightarrow$	<code>\to</code>	Function/implication arrow
$\leftarrow$	<code>\l</code>	Reverse arrow
$\leftrightarrow$	<code>\iff</code>	If and only if
$\forall$	<code>\fa</code>	For all
$\exists$	<code>\ex</code>	There exists
$\sqcap$	<code>\sqcap</code>	Meet (inf)
$\sqcup$	<code>\sqcup</code>	Join (sup)
$\perp$	<code>\bot</code>	Bottom
$\top$	<code>\top</code>	Top
$\in$	<code>\in</code>	Set membership
$\notin$	<code>\notin</code>	Non-membership
α	<code>\a</code>	Greek alpha
β	<code>\b</code>	Greek beta
$\mathbb{N}$	<code>\N</code>	Natural numbers
$\wedge$	<code>\and</code>	Logical and
$\vee$	<code>\or</code>	Logical or
$\neg$	<code>\n</code>	Logical not

Table 10.2: Lean 4 Unicode inputs used in this book.

Selected Solutions

This appendix contains solutions to the exercises that have a single definitive Lean 4 answer. Exercises asking for diagrams, written explanations, or open-ended extensions are left to the reader.

Chapter 1

Exercise 1.1 (Reflexivity and symmetry)

```
1 theorem symm_antisymm_eq {α : Type} (r : Rel α)
2   (hs : Symmetric r) (ha : Antisymm r) :
3     ∀ a b : α, r a b → a = b := by
4   intro a b h
5   -- hs gives us r b a from r a b.
6   -- ha then gives a = b from r a b and r b a.
7   exact ha a b h (hs a b h)
```

Exercise 1.3 (Product order)

```
1 instance [OrderBook.PartialOrder α] [OrderBook.PartialOrder β] :
2   OrderBook.PartialOrder (α × β) where
3   le p q := PartialOrder.le p.1 q.1 ∧ PartialOrder.le p.2 q.2
4   le_refl := fun ⟨a, b⟩ =>
5     ⟨PartialOrder.le_refl a, PartialOrder.le_refl b⟩
6   le_antisymm := fun ⟨a1, b1⟩ ⟨a2, b2⟩ ⟨h1, h2⟩ ⟨h3, h4⟩ => by
7     -- Antisymmetry component-wise, then combine with congr
8     have ha := PartialOrder.le_antisymm _ _ h1 h3
9     have hb := PartialOrder.le_antisymm _ _ h2 h4
10    simp_all
11  le_trans := fun ⟨a1, b1⟩ ⟨a2, b2⟩ ⟨a3, b3⟩ ⟨h1, h2⟩ ⟨h3, h4⟩ =>
12    ⟨PartialOrder.le_trans _ _ _ h1 h3,
13      PartialOrder.le_trans _ _ _ h2 h4⟩
```

Chapter 2

Exercise 2.1 (Monotone composition)

Antitone means order-reversing: $a \leq b \rightarrow f(b) \leq f(a)$. Composing two antitone maps reverses the order twice, yielding a monotone map. A concrete example: negate twice on $\mathbb{Z}$.

```

1  -- f and g are both antitone (order-reversing) on Int
2  def f₁ : Int → Int := fun x => -x
3  def g₁ : Int → Int := fun x => -x
4
5  -- Their composition g ∘ f is monotone
6  example : Monotone (g₁ ∘ f₁) where
7    map_le := fun a b h => by
8      -- g₁ ∘ f₁ x = -(-x) = x, so it is the identity
9      simp [g₁, f₁]
10     exact h

```

The general proof that antitone ∘ antitone = monotone:

```

1  theorem antitone_comp_antitone {α β γ : Type}
2    [LE α] [LE β] [LE γ]
3    [PartialOrder α] [PartialOrder β] [PartialOrder γ]
4    {f : α → β} {g : β → γ}
5    (hf : ∀ a b : α, a ≤ b → f b ≤ f a)
6    (hg : ∀ a b : β, a ≤ b → g b ≤ g a) :
7    Monotone (g ∘ f) where
8    map_le := fun a b h =>
9      -- a ≤ b → f b ≤ f a (hf reverses)
10     -- f b ≤ f a → g (f a) ≤ g (f b) (hg reverses again)
11     hg (f b) (f a) (hf a b h)

```

Exercise 2.2 (Fixed points)

We prove by induction that $f^n(a) \leq f^{n+1}(a)$ for all n , which shows the chain $a \leq f(a) \leq f^2(a) \leq \dots$ is ascending.

```

1  -- f^n a means applying f exactly n times to a
2  def iter {α : Type} (f : α → α) : Nat → α → α
3    | 0,      x => x
4    | n + 1, x => f (iter f n x)
5
6  theorem iter_chain {α : Type} [LE α] [PartialOrder α]
7    {f : α → α} (hf : Monotone f)
8    {a : α} (ha : a ≤ f a) :
9    ∀ n : Nat, iter f n a ≤ iter f (n + 1) a := by
10   intro n
11   induction n with

```

```

12 | zero =>
13   -- Base case: iter f 0 a = a ≤ f a = iter f 1 a
14   exact ha
15 | succ n ih =>
16   -- Inductive step: iter f n a ≤ iter f (n+1) a
17   -- so f (iter f n a) ≤ f (iter f (n+1) a) by monotonicity
18   exact hf.map_le _ _ ih

```

Exercise 2.3 (Identity is monotone)

```

1 theorem Monotone.id {α : Type} [LE α] [PartialOrder α] :
2   Monotone (id : α → α) where
3   map_le := fun _ _ h => h

```

The proof is trivial: `id a = a`, so `id a ≤ id b` is just `a ≤ b`, which is exactly the hypothesis.

Chapter 3

Exercise 3.1 (Interval partial order)

The information order on intervals: $[l_1, h_1] \leq [l_2, h_2]$ iff $l_2 \leq l_1$ and $h_1 \leq h_2$ (the first interval is contained within the second, i.e. more specific). Proof fields in the `Interval` structure are handled by proof irrelevance in `le_antisymm`: once $l_1 = l_2$ and $h_1 = h_2$ are established, both `l_le_h` fields have the same type and `congr~1` discharges the equality automatically.

```

1 structure Interval where
2   l      : Int
3   h      : Int
4   l_le_h : l ≤ h
5
6 instance : LE Interval where
7   -- [l1, h1] ≤ [l2, h2] iff l2 ≤ l1 ∧ h1 ≤ h2
8   -- (first interval is contained within / more specific than second)
9   le := fun i j => j.l ≤ i.l ∧ i.h ≤ j.h
10
11 instance : PartialOrder Interval where
12   le_refl := fun a =>
13     (Int.le_refl a.l, Int.le_refl a.h)
14
15   le_antisymm {a b} := fun hyp1 hyp2 => by
16     -- hyp1 : b.l ≤ a.l ∧ a.h ≤ b.h

```

```

17   -- hyp2 : a.l ≤ b.l ∧ b.h ≤ a.h
18   cases a; cases b
19   rename_i l1 h1 l1_le_h1 l2 h2 l2_le_h2
20   congr 1
21   · exact Int.le_antisymm hyp2.left hyp1.left   -- l1 = l2
22   · exact Int.le_antisymm hyp1.right hyp2.right  -- h1 = h2
23   -- l_le_h field closed automatically by proof irrelevance
24
25   le_trans {a b c} := fun hyp1 hyp2 => by
26     rcases a with ⟨a_l, a_h, a_le⟩
27     rcases b with ⟨b_l, b_h, b_le⟩
28     rcases c with ⟨c_l, c_h, c_le⟩
29     -- lower bound chain: c_l ≤ b_l ≤ a_l (reversed – more specific = smaller
30     -- ↪ l)
31     have hl_ba : b_l ≤ a_l := hyp1.left
32     have hl_cb : c_l ≤ b_l := hyp2.left
33     -- upper bound chain: a_h ≤ b_h ≤ c_h (forward – more specific = smaller
34     -- ↪ h)
35     have hh_ab : a_h ≤ b_h := hyp1.right
36     have hh_bc : b_h ≤ c_h := hyp2.right
37     exact ⟨Int.le_trans hl_cb hl_ba, Int.le_trans hh_ab hh_bc⟩

```

Exercise 3.3 (Characterising the order)

For the forward direction, antisymmetry closes the goal from $a \sqcup b \leq b$ (both $a \leq b$ and $b \leq b$ hold, so b is an upper bound) and $b \leq a \sqcup b$ (from `sup_le_right`). For the backward direction, `h : a  $\sqcup$  b = b` is used backwards with `rw [← h]` to rewrite the goal $a \leq b$ as $a \leq a \sqcup b$, which is exactly `sup_le_left`.

```

1  theorem le_iff_sup_eq {α : Type} [Lattice α] {a b : α} :
2    a ≤ b ↔ a  $\sqcup$  b = b := by
3    apply Iff.intro
4    · intro h
5      apply PartialOrder.le_antisymm
6      · apply Lattice.le_sup
7        · exact h
8        · exact PartialOrder.le_refl b
9      · exact Lattice.sup_le_right a b
10   · intro h
11     -- h : a  $\sqcup$  b = b; rw [← h] rewrites goal `a ≤ b` to `a ≤ a  $\sqcup$  b`
12     rw [← h]
13     exact Lattice.sup_le_left a b

```

Exercise 3.4 (The product lattice)

Every axiom reduces to the corresponding axiom applied independently to each component. `le` is defined inline so `le\_antisymm` can use `rcases` to name the components and `congr~1` to split the pair equality into two component equalities.

```

1 instance {α β : Type} [Lattice α] [Lattice β] : Lattice (α × β) where
2   le           := fun a b => (a.fst ≤ b.fst) ∧ (a.snd ≤ b.snd)
3   le_refl      := fun a => by
4     apply And.intro
5     · apply PartialOrder.le_refl
6     · apply PartialOrder.le_refl
7   le_antisymm {a b} := fun hyp1 hyp2 => by
8     rcases a with ⟨l1, r1⟩
9     rcases b with ⟨l2, r2⟩
10    congr 1
11    · exact PartialOrder.le_antisymm hyp1.left hyp2.left
12    · exact PartialOrder.le_antisymm hyp1.right hyp2.right
13  le_trans {a b c} := fun hyp1 hyp2 => by
14    apply And.intro
15    · exact PartialOrder.le_trans hyp1.left hyp2.left
16    · exact PartialOrder.le_trans hyp1.right hyp2.right
17  inf         := fun a b =>
18    ⟨Lattice.inf a.fst b.fst, Lattice.inf a.snd b.snd⟩
19  sup         := fun a b =>
20    ⟨Lattice.sup a.fst b.fst, Lattice.sup a.snd b.snd⟩
21  inf_le_left  := fun a b => by
22    apply And.intro
23    · apply Lattice.inf_le_left
24    · apply Lattice.inf_le_left
25  inf_le_right := fun a b => by
26    apply And.intro
27    · apply Lattice.inf_le_right
28    · apply Lattice.inf_le_right
29  le_inf       := fun a b c hyp1 hyp2 => by
30    apply And.intro
31    · apply Lattice.le_inf
32    · exact hyp1.left
33    · exact hyp2.left
34    · apply Lattice.le_inf
35    · exact hyp1.right
36    · exact hyp2.right
37  sup_le_left  := fun a b => by
38    apply And.intro
39    · apply Lattice.sup_le_left
40    · apply Lattice.sup_le_left
41  sup_le_right := fun a b => by
42    apply And.intro

```

```
43     · apply Lattice.sup_le_right
44     · apply Lattice.sup_le_right
45 le_sup      := fun a b c hyp1 hyp2 => by
46   apply And.intro
47     · apply Lattice.le_sup
48     · exact hyp1.left
49     · exact hyp2.left
50     · apply Lattice.le_sup
51     · exact hyp1.right
52     · exact hyp2.right
```

Further Reading

- **Davey & Priestley**, *Introduction to Lattices and Order*, 2nd ed., Cambridge University Press, 2002. The standard introductory text on lattice theory, accessible and rigorous.
- **Radul & Sussman**, *The Art of the Propagator*, MIT CSAIL Technical Report, 2009. The foundational paper for the propagator model used in this book.
- **Tate, Stepp, Lerner & Chambers**, *Equality Saturation: a New Approach to Optimization*, POPL 2009. An application of lattice-based fixed-point algorithms to compiler optimization.
- **Nielson, Nielson & Hankin**, *Principles of Program Analysis*, Springer, 1999. Covers dataflow analysis (a lattice-based framework) and abstract interpretation.
- **The Lean 4 Documentation**, <https://lean-lang.org>. Official reference for Lean 4 syntax, tactics, and the core library.
- **Mathlib4**, https://leanprover-community.github.io/mathlib4_docs/. The community mathematics library for Lean 4. Once you have finished this book, exploring `Mathlib.Order` will show you how the constructions here scale to a large, verified library.